Practical Attacks on Session Messenger and Oxen Blockchain

Tingfeng Yu
School of Computing
Australian National University
Canberra, Australia
Tingfeng.Yu@anu.edu.au

Thomas Haines
School of Computing
Australian National University
Canberra, Australia
Thomas.Haines@anu.edu.au

***Abstract*—Session is a decentralised secure (anonymous) messenger that combines onion routing with the Oxen Proof-of-Stake blockchain to provide metadata-private communication. Our study presents the first comprehensive analysis of Session's messaging protocol and its integration with the Oxen blockchain. In analysing Session and the underlying Oxen blockchain, we uncovered seven vulnerabilities.**

Most notably we discovered flaws in the Oxen consensus protocol which could allow network takeover in a realistic setting, thereby undermining the integrity guarantees on which Session's anonymity layer depends. We also discovered serious vulnerabilities in Version 1 of Session's group chat protocol. We conducted extensive simulations to analyse the impact of these vulnerabilities and provide recommendations to reinforce both the Oxen protocol and the Session client to mitigate these attacks.

***Index Terms*—Session, Oxen, Proof-of-Stake, Blockchain, anonymity, Onion Routing, secure messengers.**

1. Introduction

The ongoing interest in secure communication has instigated the development of various protocols and applications to provide robust security guarantees under different settings. Some widely adopted protocols such as the Signal Protocol, including the subprotocols X3DH [1], Double Ratchet [2], and Sesame [3], have been analysed extensively by past research [4]–[6], and their end-to-end guarantees (confidentiality, authenticity, forward and post-compromise secrecy) are now fairly well understood.

However, their threat models implicitly assume an adversary who can intercept or tamper with traffic, but:

- either gains no value in identifying the patterns of communication, or
- cannot observe the entire network.

In practice, state–level surveillance [7] and large-scale traffic correlation by vendors [8] already stretch this assumption. A global passive adversary who can eavesdrop on every link in the network can gain valuable information solely from the metadata associated with the end-to-end encrypted message content. Protecting metadata therefore demands protocols that offer an additional notion of security, metadata privacy or anonymity, in addition to the content-layer security.

Session [9] is an instant messenger developed by the Oxen Project in 2020. It aims to meet this stronger goal by leveraging a decentralised onion routing network to minimise metadata exposure. Session's onion network relies on Oxen, a Proof-of-Stake (PoS) blockchain, as an append-only public ledger that reflects network changes, e.g., new nodes joining the network, nodes being removed from the network, aiming to provide a resilient and transparent view of the network; a security compromise of Oxen can be exploited to break anonymity in Session.

Despite having an open-source codebase and millions of users [10], Session and its underpinning Oxen blockchain still lack an up-to-date, systematic description of its protocols, and have never been subjected to peer-reviewed study. The public whitepapers for Session and Oxen omit implementation details and have not been revised to reflect protocol changes introduced by later hard-forks; also missing are detailed security claims and argumentation. The lack of existing analysis, esoteric design, and large user base make it an interesting object of study. The only publicly available assessment of Session to date was a security audit commissioned from Quarkslab in May 2021 [11], where auditors examined the Session client implementation for different platforms. However, the analysis was limited to the client software and focused on classical application security issues, such as local data protection and certificate handling. Crucially, assessment of Session's protocol design was out of scope of the assessment. We aim to close these gaps by providing the first protocol security assessment of the Session and the Oxen blockchain and uncover vulnerabilities hidden from pure application testing.

1.1. Contributions

We present the first security analysis of Session and its integration with Oxen. Our main contributions are:

Comprehensive view of Session's protocol stack (§3)
We extract and formally document the core protocols of Session and Oxen, filling numerous gaps in the public whitepaper.

Four consensus-layer vulnerabilities in Oxen (§4)

- *OV1 and OV2. Quorum bias:* We discovered that two Oxen quorums (Pulse for block production, and Obligation for performance monitoring) are mutually influential.
  OV1 and OV2 show how an attacker-dominated quorum can deterministically skew the membership of the other quorum in the very next block, snowballing its control over future rounds.

- *OV3. Insufficient vote validation:* The network accepts duplicate, early or late quorum votes, allowing malicious quorums to create repeated or premature/overmature network state changes to its advantage, vastly exacerbating OV1 and OV2.
- *OV4. Immediate deregistration:* missing protocol integrity checks allow malicious obligation quorums to bypass the intended pipeline for punishing under-performing nodes, by irreversibly removing honest nodes from the network.

We combine OV1–OV4 into practical network-takeover attacks and evaluate their feasibility under both simulation and emulation (§6), showing that $\leq 30\%$ adversarial stake suffices to control the entire network within ≈ 4 days; should such an attack occur it would force the system to hard-fork to recover, possibly irreversibly undermining trust in the system.

Three client-level attacks on Session (§5)

- *SV1. Seed-node bias:* Session clients rely on seed nodes to learn the onion routing network. We discovered that a single malicious seed node can supply a fully adversarial service-node list to clients, subverting onion-path anonymity.
- *SV2. Inadequate rekeying:* In group chat protocol version one, an incomplete re-key procedure allows departed members to read future messages.
- *SV3. Closed-group privilege escalation:* In group chat protocol version one, inadequate access control lets any member impersonate the sole admin when inviting new users; in the current protocol version (version two), we observe a weaker privilege escalation where an attacker can impersonate admin actions in members' UI, though without altering actual group state.

Mitigations (§7) We propose changes which either patch or partially mitigate the vulnerabilities in Oxen and Session; unfortunately, in several cases a complete patch is not possible because the vulnerability is inherent to the design.

1.2. Responsible disclosure

All confirmed vulnerabilities were privately disclosed to the Oxen developers on the 16th of July 2025, along with outlined mitigations that eliminate the demonstrated exploits. At the time of writing, the developers have implemented mitigations which are currently deployed on the testnet and staged for release to the mainnet.

1.3. Paper Structure

In Section 2 we provide the necessary preliminary knowledge on relevant constructions (onion routing and blockchain) and survey literature on secure messengers. In Section 3, we present the protocol details for the implementation of Session and Oxen. Section 4 and Section 5 detail the seven protocol-level and consensus-level vulnerabilities we uncover. Section 6 shows how these consensus-level vulnerabilities compose into practical network-takeover attacks and evaluates their impact through both simulated and emulated experiments. Section 7 presents defences for each identified vulnerability, and Section 8 concludes our work.

2. Background

This section covers both preliminary background material for the techniques used in Oxen and Session, and related work on other secure messengers.

2.1. Preliminaries

Session and Oxen combine several common techniques in unusual ways. Our preliminaries will first cover the relevant techniques related to onion routing before covering those related to blockchain.

2.1.1. Onion Routing. Onion routing [12] is a classic approach to achieve sender–receiver unlinkability by layering secure public-key encryption with multi-hop forwarding. A client who wishes to message `dest` picks a random path of k relay nodes $h_1, \ldots, h_k$ and wraps the payload in k nested encryptions:

$$c_k = \mathsf{Enc}_{pk_{h_k}}\left(\mathtt{addr}_{\mathtt{dest}}, m\right)$$
$$c_{i-1} = \mathsf{Enc}_{pk_{h_{i-1}}}\left(\mathtt{addr}_{h_i}, c_i\right) \quad \text{for } i = 2 \ldots k$$

with c_0 sent to N_1. Each relay N_i uses its private key to decrypt c_i, learns the next hop $addr_{N_{i+1}}$, and forwards c_{i+1} onwards; the final relay then passes the message m, which is normally also encrypted, to the destination.

Early designs of onion routing assumed a known and static set of relays, leaving the peer discovery process outside the protocol model. However, it has become an important component in deployed onion routing networks, as network topology and node availability likely change over time. Modern implementations such as Tor [13] rely on trusted directory authorities to publish relay information and maintain network consensus.

Onion routing provides effective protection against a local or partially-observing adversary, but breaks down under a global passive attacker who can observe the traffic on every link in the network, e.g., ($\mathtt{Alice} \rightarrow N_1$, $N_1 \rightarrow N_2 \rightarrow \mathtt{Bob}$). Leveraging timing correlations between outgoing and incoming packets of every node in the network allows them to infer the origin and destination of a message despite the layered encryption and relaying.

To mitigate simple intersection or timing attacks, many onion networks may inject dummy/cover messages at random intervals, so that an observer cannot tell which packets carry real payloads (as in modern Tor clients).

2.1.2. Blockchain Fundamentals. A blockchain is an append-only, distributed ledger in which every block contains a hash pointer to its predecessor, forming a chain. Following Nakamoto's foundational Bitcoin proposal [14], blockchains have been generalised into many settings. A secure blockchain offers two key properties under partially-synchronous network and Byzantine faults [15]:

- Consistency (Safety): Honest nodes eventually agree on the same prefix of blocks; no two honest nodes will indefinitely maintain conflicting histories.
- Liveness: New, valid transactions submitted by honest clients will eventually be accepted and finalised in the ledger; the chain will not stall forever.

Achieving both properties under realistic network conditions, with message delays, dropped packets, and potentially malicious actors, is what consensus protocols

like Proof-of-Work (e.g. Bitcoin) or Proof-of-Stake (e.g. Ouroboros) are designed to guarantee.

Consensus definitions. The consensus mechanism is a fundamental component of a blockchain. It defines how a distributed network agrees on the next block in the presence of Byzantine faults like message delays and active adversaries. Consensus mechanisms in distributed systems are often analysed in the Byzantine fault tolerant (BFT) model, where nodes may behave arbitrarily due to faults or adversarial control. Classical results show that deterministic consensus can tolerate at most f misbehaving nodes among $3f + 1$ participants [16], [17]. Therefore, classical BFT protocols tolerate fewer than one third Byzantine participants. However, blockchain protocols rely on different models and may aim to tolerate different adversarial thresholds. There are two widely used paradigms for consensus mechanisms, Proof-of-Work (PoW) and Proof-of-Stake (PoS).

Proof-of-Work was introduced by Nakamoto in Bitcoin [14], PoW ties voting power to computational effort: the more hash rate a node controls, the greater its probability of mining the next block. While this approach has proven robust, it carries two well-known drawbacks:

- the energy cost of sustained hash computation, and
- a tendency towards centralisation as economies of scale favour large mining pools.

Proof-of-Stake was proposed on the Bitcoin Forum [18] as a natural alternative to the energy-intensive mining of PoW. It was subsequently implemented in PeerCoin [19] and formalised in later works [20], [21]. In PoS blockchains, voting power is proportional to the stake of a participant rather than computational power; the definition of stake varies between implementations but the idea is that the validators' voting power is constrained by their share of the currency and if they misbehave they can be financially penalised. By shifting the resource from computing power to capital at risk, PoS aims to broaden participation, reduce energy consumption, and mitigate miner centralisation. Part of the success of PoS blockchains could be attributed to the fact that entities who are capable of mounting attacks must hold significant stake, and the stake value is tied to the reputation of the system, which would be undermined by attacks.

2.1.3. Proof-of-Stake Mechanics. Proof-of-Stake (PoS) uses a stake-weighted lottery, meaning that the probability of participants being elected to create the next block is proportional to the stake they own. A PoS blockchain needs to solve the following problems:

Committee election. *How to sample block proposers and validators based on stake?* An example is the VRF-based self-sortition used by Algorand [22] and Ouroboros Praos [23], where each node self-checks eligibility with a sortition function. Let s denote the user's stake, S the total stake, R the public randomness derived from the current chain state. the sortition function outputs a hash v_{sel}, a proof π_{sel}, and a selection result j_{sel}:

$$(v_{\text{sel}}, \pi_{\text{sel}}, j_{\text{sel}}) \leftarrow \mathsf{Sortition}_{\text{sk}}(s, R, S).$$

The Sortition function has two layers:

- Verifiable Random Function (VRF): A VRF [24] is a pseudorandom function with a publicly verifiable proof of correctness. Sortition algorithm evaluates a VRF on the public randomness R, produces a hash and proof: $(v_{\text{sel}}, \pi_{\text{sel}}) \leftarrow \mathsf{VRF}_{sk}(R)$. The proof allows anyone to verify the hash using the public key corresponding to sk but does not allow the adversary to predict ahead of time what effect the randomness will have on the honest user's selection.
- Stake-Proportional Selection: An algorithm that produces a selection outcome j_{sel} given the VRF output, with the likelihood of selection proportional to the user's stake $\Pr[j_{\text{sel}} > 0] \propto s/S$. A node is eligible in the round iff $j_{\text{sel}} = 1$.

Honest nodes reveal their eligibility by broadcasting $(v_{\text{sel}}, \pi_{\text{sel}}, j_{\text{sel}})$. Because the VRF output is unpredictable, and the threshold depends only on the public stake ratio s/S, the election is provably fair as no adversary can bias v_{sel} after seeing the public randomness.

Randomness maintenance. *How to generate unbiased public randomness for future committee election?* The security of a PoS blockchain relies on its randomness, which must be *unpredictable* until the current block is final and *bias-resistant* such that no adversary can skew the randomness to gain advantages in future committee selections. Randomness maintenance in deployed PoS blockchains (e.g., Praos and Algorand) often involves two scales: per-block randomness produced by each block proposer, and the public randomness aggregated from a window of per-block randomness. The latter is used for committee selection.

2.2. Related Work

Messaging systems without extensive analysis are prone to security vulnerabilities, even when they are deployed in applications used by millions of users globally; we discuss a few examples. Signal's double-ratchet and X3DH handshake protocols have been formalised and proven to provide strong guarantees such as forward and post-compromise secrecy [25], [26]. However, implementation analysis of the Signal Android client and server code from [27] discovered that the notion of forward secrecy and future secrecy are too weak at a code level, despite both properties being proven to hold in theoretical evaluation of the protocol. Additionally, researchers discovered that the Signal app does not have one of its documented features: the Sesame algorithm, a session management algorithm developed to extend the Signal Protocol to a multi-device setting [3], was not fully implemented in the app as the session recovery mechanism was missing. Similarly, reverse-engineering and formal analysis work on Threema, a proprietary secure messenger, uncovered design flaws that enable message duplication, denial-of-service, and key-recovery attacks [28].

Another strand of research on secure messengers focuses on anonymity. While most secure messengers aim to provide strong security guarantees to protect the message content, the network-level metadata associated with these packets often reveals sensitive information, such as the sender and receiver information (e.g., IP address), and other details that can potentially be used to infer the nature of the conversations, or the sender-receiver identity. Several deployed messengers have taken network-level privacy into their security goals:

- Bridgefy [29] uses a mesh-network design to facilitate decentralised messaging;
- xxMessenger [30] uses cMix [31], [32], a low latency mixnet as its anonymity layer;
- Briar [33], Ricochet [34], and Session [9] use onion routing.

Tor [13], the most widely deployed onion-routing network, has influenced the design of several instant messengers. Both Ricochet and Briar use Tor to provide metadata privacy. However, Tor's peer discovery process relies on a small set of trusted directory authorities which raises concerns about centralisation and single points of failure. Motivated by these concerns, the Loki Foundation (later rebranded to the Session Foundation) proposed the LokiNet, a custom onion routing network for Session [35], to replace directory servers with the Oxen proof-of-stake blockchain, where Oxen provides a decentralised public record of the onion relay network state.

Despite the widely recognised importance of metadata hiding in the field of anonymity research, systems that promise metadata privacy remain relatively understudied. Among the four metadata-private messengers listed above, only Briar and Bridgefy have received published security analyses. Song's study of Briar [36] provides the first comprehensive documentation of its bespoke protocol stack, while Albrecht et al. [37] reverse-engineered Bridgefy's Bluetooth-mesh protocol and uncovered multiple flaws that break the application's privacy claims.

However, findings of these works are closely tied to the design of the respective systems. Session differs substantially in its architecture, and therefore insights from these studies are not directly transferable. Session is not the first messenger that relies on Onion-routing, but the first that uses a blockchain consensus layer to govern the routing infrastructure. To our knowledge, Oxen's proof-of-stake consensus has not previously been studied. Vulnerabilities at this layer could allow adversaries to manipulate quorum decisions and influence the composition of the routing network itself, fundamentally undermining the privacy of Session. To bridge the gap, we provide the first in-depth analysis of Session and Oxen, focusing on the quorum mechanisms that determine the set of service nodes operating the onion routing network.

3. Implementation Analysis

In this section, we first describe what security claims Sessions and Oxen make, before presenting the core protocols of both systems as implemented. We begin with Session's decentralised architecture: its end-to-end encrypted messaging protocol, swarm-based storage, and onion-routing network. We then turn to Oxen, the PoS blockchain that underpins Session's integrity.

3.1. Security Properties

Ratings of secure messengers are a common topic in blog posts, published by various companies and media outlets; such rankings often rate Session very highly for security [38]–[41]. However, to the best of our ability to tell Session makes only a few informal security claims and no formal (in the academic sense) claims. It is possible to infer certain security objectives from the design decisions and by analogy to other PoS blockchains.

Before introducing these security properties, we first explain the role of Oxen blockchain in relation to Session. Session uses an onion routing network to provide metadata privacy, which relies on the Oxen blockchain to maintain a decentralised and consistent view of active peers/service nodes and to incentivise the peers to participate. This is similar to the blockchain network underlying the deployment of cMix [30]. While the developers do claim that "By using this network, Session doesn't have a central point of failure, and Session's creators have no capacity to collect or store personal information about people using the app." [42] the references in the white paper focus on the financial incentive side more than security [43]. Our analysis, particularly vulnerability SV1 (5.1), suggests that the Session approach does not offer any advantages over the centralised directory service approach taken by Tor.

In this study, we focus on the security properties of Oxen's consensus mechanisms for Pulse and Obligation quorum (discussed later), both quorums produce decisions that affect service node state. Since Oxen does not appear to publish a formal security model for either quorum, we extract the following intended properties from the design rationale in oxen-improvement-proposals, and the implementation oxen-core:

P1 Deterministic quorum selection: Given the same chain state, all nodes deterministically derive the same quorum composition.

P2 Threshold authorisation: A quorum decision is valid only if it is authorised by the protocol-defined threshold of members.

P3 Temporal quorum authority: A quorum can exercise its authority only for its designated round, height, or bounded validity window.

P4 Quorum selection fairness: A party's influence over quorum selection outcome should be determined only by its stake fraction and the distribution induced by protocol randomness, it cannot be increased by deviating from honest execution.

Oxen service nodes may undergo state changes authorised by Obligation quorums, which implies:

P5 State change integrity: Every accepted state change must represent a legal transition given the service node's state.

3.2. Methodology

We studied Session and its surrounding stack based on the official specification and implementations:

- `session-android` (commit: bdb6e7d and e4c16ae): the Android client of Session. Commit `bdb6e7d` corresponds to the group chat protocol v1 (the version current at the start of our study), while `e4c16ae` implements v2, which we also examined to assess the current state of the protocol.
- `oxen-core` (hard fork: 22, commit: 55f6f10): implementation of the Oxen blockchain protocol, including the consensus protocol, transaction validation, and peer-to-peer communication with service node management (e.g., swarm updates).

Our methodology combined specification review with an implementation-level audit. We conducted a manual code review of the Session and Oxen repositories to extract the underlying protocols, cross-checking them against the official specifications and reconstructing missing/outdated details; some properties of the protocol only become apparent during our emulation and simulation. To further our analysis and provide empirical proof of our findings:

- we developed a patched Session client for validating attacks on group chat;
- we deployed a local testnet to validate the presence of several Oxen protocol vulnerabilities;
- we implemented a lightweight simulation of Oxen, parameterised to reflect mainnet conditions, to demonstrate the practical exploitability of network-takeover attacks (Section 6).

3.3. Session

In Session, messages are relayed and stored by a distributed onion-routing network of Service Nodes. Each Service Node fulfills three roles:

- *Onion router:* forwards layered (onion) encryption packets across three-hop paths.
- *Storage server:* stores offline messages for its assigned users.
- *Blockchain worker:* validates Oxen consensus and records state changes, which in turn drive swarm rebalancing and node-register/deregister events.

Service Nodes are logically grouped into *swarms* for message storage. Each swarm has 5–9 Service Nodes. Oxen's deterministic, block-triggered swarm assignment algorithm uses a pseudorandom generator seeded by the block's hash to rebalance underfull (< 5 nodes) and overfull (≥ 10 nodes) swarms toward the ideal size of seven. All incoming Session messages are redundantly distributed across nodes in that swarm.

Figure 1 defines the key notations and data types for later references. Each service node maintains a local view of the network state $S : \texttt{State}$. In addition, we use $\texttt{sns}_\texttt{a}$ and $\texttt{sns}_\texttt{d}$ to represent the list of active service node (nodes that are currently participating in the network) and decommissioned service nodes (nodes that have failed performance checks but have not yet been deregistered from the system), where $S.\texttt{sns} = \texttt{sns}_\texttt{d} \bigcup \texttt{sns}_\texttt{a}$ and $\emptyset = \texttt{sns}_\texttt{d} \bigcap \texttt{sns}_\texttt{a}$. We now illustrate Session's core protocol by tracing the workflow from onboarding to sending the first end-to-end message.

3.3.1. Onboarding. A new client generates a random `seed`, from which it derives two long-term key pairs:

- An Ed25519 signing key pair $(sk_s^{\text{id}}, pk_s^{\text{id}})$, computed from the padded seed. The public key pk_s serves as the user's Session ID.
- An X25519 keypair $(sk_s^{\text{enc}}, pk_s^{\text{enc}})$, derived from the Ed25519 key pair via Curve25519 conversion, is used as the client's identity key within libsignal (implementing the end-to-end encryption part of the Signal protocol) for end-to-end encryption.

To use Session's onion routing network, a new client would also fetch the $\texttt{sns}_\texttt{a}$ via one or more seed nodes (a hard-coded service node that bootstraps network entry); the exact details lead to a vulnerability (Section 5.1).

3.3.2. Contact Discovery. To send a message, the sender must first obtain the recipient's public identity key pk_r^{id} and encryption key pk_r^{enc} out-of-band.

3.3.3. Message Sending. The complete message-sending flow is illustrated in Figure 2, which corresponds to the five steps described below.

1) *E2E encryption:* Let pk_s^{id} be the sender's signing key/Session ID and pk_r^{enc} the recipient's encryption key. The sender binds the message to itself and the recipient by signing

$$\sigma \leftarrow \text{Sign}_{sk_s^{\text{id}}}\big(m \| pk_s^{\text{id}} \| pk_r^{\text{enc}}\big).$$

Form the payload $p \leftarrow m \| pk_s^{\text{id}} \| \sigma$, then E2E encrypt p with $\text{Enc}(\cdot)$ (Algorithm 1):

$$c_{\text{e2e}} \leftarrow \text{Enc}(pk_r^{\text{enc}}, p),$$

where $\text{Enc}(pk, p)$ denotes a hybrid encryption that takes a payload p and an X25519 public key pk as input. It first generates an ephemeral X25519 keypair; then derives a symmetric key from ECDH and HMAC, and encrypts the payload with AES-GCM.

Algorithm 1: $\mathsf{Enc}(pk, p)$

Output: $c \| epk$: ciphertext c and ephemeral public key epk

Operation:

```
(esk, epk) ← X25519.EphemeralKeyGen()
k ← HMAC_"LOKI"(ECDH(esk, pk))
c ← AES-GCM.Enc_k(p)
return c||epk
```

2) *Peer Swarm Discovery:* To contact a recipient, the client must resolve the recipient's Session ID pk_r^{id} to the recipient's swarm. The client submits pk_r^{id} to a Service Node in $\texttt{sns}_\texttt{a}$ via the $\texttt{getSwarm}(pk_r^{\text{id}})$ API. The node deterministically maps $pk_r^{\text{id}} \mapsto id_{\text{swarm}}$ and returns the swarm description $\texttt{swarm}_r$, i.e., a set of Service Nodes specified by onion address, port, and public key:

$$\texttt{swarm}_r : \texttt{SNode}[] \leftarrow \texttt{getSwarm}(pk_r^{\text{id}})$$

3) *Onion construction:* The client selects three random Service Nodes (h_1, h_2, h_3) and one destination node d from the recipient's swarm, then constructs the onion-encrypted ciphertext by wrapping the E2E encrypted payload ($p_{\text{e2e}} \leftarrow pk_r^{\text{id}} \| c_{\text{e2e}}$) outward, from the destination node to the guard, by repeated applications of $\text{Enc}(\cdot)$ (Algorithm 1). The resulting layered onion construction is as follows:

$$c_d \leftarrow \text{Enc}(pk_d^{\text{enc}}, p_{e2e}),\ p_d = pk_d^{\text{id}} \| c_d$$
$$\dots$$
$$c_{h_1} \leftarrow \text{Enc}(pk_{h_1}^{\text{enc}}, p_2),\ \text{onion} = c_{h_1}$$

4) *Relay:* The onion is sent to the guard h_1; each hop h_i peels off its layer using the encryption secret key, revealing the next hop (addressed by its public key $pk_{h_{i+1}}^{\text{id}}$) and the remaining onion. The peeled onion is then forwarded along the path until the destination node, which fully decrypts the innermost payload,

- $\texttt{SNode} := \{pk, \texttt{lastPulse}, \texttt{active}, \ldots\}$: a service node record consisting of the node's public key, the block height (lastPulse) at which it last participated in Pulse (see Sec.3.4.1), and its activeness $\texttt{active} \in \{\top, \bot\}$.
- $\texttt{Block} := \{\texttt{blockHash}, \texttt{pulseRand}, \texttt{txs}, \ldots\}$: a blockchain block containing its hash, a randomness field, and transaction references.
- $\texttt{OQ} := \{V, W\}$: an Obligation quorum, containing the validators' public keys V, workers' public keys W.
- $\texttt{PQ} := \{p, V\}$: a Pulse quorum, containing the block producer's key p and validators' keys V.
- $\texttt{StateChange} \in \{\texttt{REG}, \texttt{DEREG}, \texttt{DECOM}, \texttt{RECOM}, \texttt{NOOP}\}$: the type of service node state transition.
- $\texttt{Tx} := \{\texttt{height}, \texttt{sc}, i_w, \texttt{votes}\}$: a state-change transaction record, where height is the cached height for the Obligation quorum, sc is the state change operation, i_w is the index of the target worker within the relevant Obligation quorum, and $\texttt{votes} = [(v_1, \sigma_1), \ldots, (v_m, \sigma_m)]$ is the list of validator indices and signatures.
- $\texttt{State} : \{\texttt{B} : \texttt{Block}[], \texttt{sns} : \texttt{SNode}[], \texttt{txs} : \texttt{Tx}[], \texttt{Q} : \texttt{OQ}[]\}$: The network state, consisting of the blockchain, service nodes in the network, pending transactions, and cached Obligation quorums.

Figure 1: Notation

obtaining the E2E encrypted message addressed to the receiver: $p_{\text{e2e}} := pk_r^{\text{id}} \| c_{\text{e2e}}$. The destination node d broadcasts the message to all nodes in the swarm, each will retain the message for retrieval.

5) *Message Retrieval:* The receiver polls its swarm for incoming messages. The requested swarm node will return any messages with a matching Session ID pk_r^{id}.

It is clear that if all three Service Nodes in the chosen path are adversarially controlled then the message sending protocol provides no anonymity; this is a common limitation of any system attempting to distribute trust but the vulnerabilities in Oxen allow the adversary to control all active Service Nodes within the threat model.

3.4. Oxen

The Oxen blockchain serves as a public ledger for the network state, where blocks are produced through quorum consensus and state changes are logged on-chain as transactions. The two quorums that are particularly relevant to this paper are the Pulse quorum for block creation, and the Obligation quorum for node activity monitoring. Table 1 shows the essential network parameters used by Oxen's mainnet and their values at time of this study.

Parameter	Value
Pulse quorum size n_{Pulse}	12
Pulse threshold k_{P}	7
Obligation validators n_v	10
Obligation workers n_w	$\mathsf{Max}(0.01 \times \lvert S.\texttt{sns}\rvert, 50)$
Obligation threshold k_{O}	7
Pulse entropy lag pulseLag	21 blocks
Obligation vote lifetime voteLife	60 blocks
Cache delay reorgBuffer	11 blocks (Appendix C.3)

TABLE 1: Network Parameters ρ

$\mathsf{Oxen}_\rho(S_i)$ models the core Oxen protocol as a local state-transition function parameterised by the network parameters ρ. Our model presents Oxen at the level of successive chain states, while omitting lower-level implementation details such as network communication and low-level data format. Appendix C.1 discusses these abstractions in more detail.

Given a service node's pre-state $S_i : \texttt{State}$, Oxen_ρ returns the post-state S_{i+1} obtained after the next accepted block is produced. The protocol proceeds as follows: First, it yields a new valid block b through $\mathsf{RunPulse}_\rho(S_i)$, which abstracts the block production process involving Pulse quorum (Sec. 3.4.1). Then, the protocol updates the state using b: it appends b to the chain, removes transactions in b from the mempool, updates the service node list, derives and caches the Obligation quorum (Sec. 3.4.2) for the new height, and runs the Obligation quorum. Any state-change transactions produced by the running Obligation quorum are validated then added to the mempool. Next, we present the details of each quorum.

Algorithm 2: $\mathsf{Oxen}_\rho(S_i)$

Input: Current state S_i
Output: Next state S_{i+1}
Operation:

1. $b \leftarrow \mathsf{RunPulse}_\rho(S_i)$ ▷ Alg. 3
2. $S_{i+1}.B \leftarrow S_i.\texttt{B} \cup \{b\}$
3. $S_{i+1}.\texttt{txs} \leftarrow S_i.\texttt{txs} \setminus b.\texttt{txs}$
4. $S_{i+1}.\texttt{sns} \leftarrow \mathsf{UpdateSNS}(S_i.\texttt{sns}, b.\texttt{txs})$ ▷ App. C.2
5. $S_{i+1}.Q[b.\texttt{height}] \leftarrow \mathsf{GenObl}_\rho(b, S_{i+1}.\texttt{sns})$ ▷ Alg. 8
6. $\texttt{txs}_{\texttt{obl}} \leftarrow \mathsf{RunObl}_\rho(S_{i+1})$ ▷ Alg. 10
7. $S_{i+1}.\texttt{txs} \leftarrow S_{i+1}.\texttt{txs} \cup \{\texttt{tx} \in \texttt{txs}_{\texttt{obl}} : \mathsf{ValidTx}(\texttt{tx}, S_{i+1})\}$ ▷ Alg. 11
8. **return** S_{i+1}

3.4.1. Pulse quorum. $\mathsf{RunPulse}_\rho(S_i)$ models the protocol's top-level behaviour for block production with Pulse consensus, the protocol runs up to 255 *rounds* (an internal counter for each height) to produce a valid block, then falls back to PoW (Proof-of-Work) if no valid block is produced. At each round, the protocol generates and runs a Pulse quorum.

Quorum Generation. $\mathsf{GenPulse}_\rho(S_i, r)$ and its helper functions illustrates the Pulse quorum generation process, which involves the following steps:

- sorts the active node list $\texttt{sns}_{\texttt{a}}$ by lastPulse to favour nodes that have waited longer;
- $\mathsf{GetPulseEntropy}_\rho(S_i.\texttt{B}, r)$ computes entropy values using randomness from a fixed window of blocks $([b_{\texttt{start}}, \ldots, b_{\texttt{end}}])$;
- $\mathsf{GetSI}_\rho(E, \lvert\texttt{sns}_{\texttt{a}}\rvert)$ maps the resulting entropy vector E to SI, a list of indices within $[0, \lfloor \frac{\lvert\texttt{sns}_{\texttt{a}}\rvert}{2} \rfloor)$;
- The indices SI are then applied directly to the sorted active node list to finalise the pulse quorum selection via $SN \leftarrow \mathsf{ApplySI}_\rho(SI, \texttt{sns}_{\texttt{a}})$. The resulting

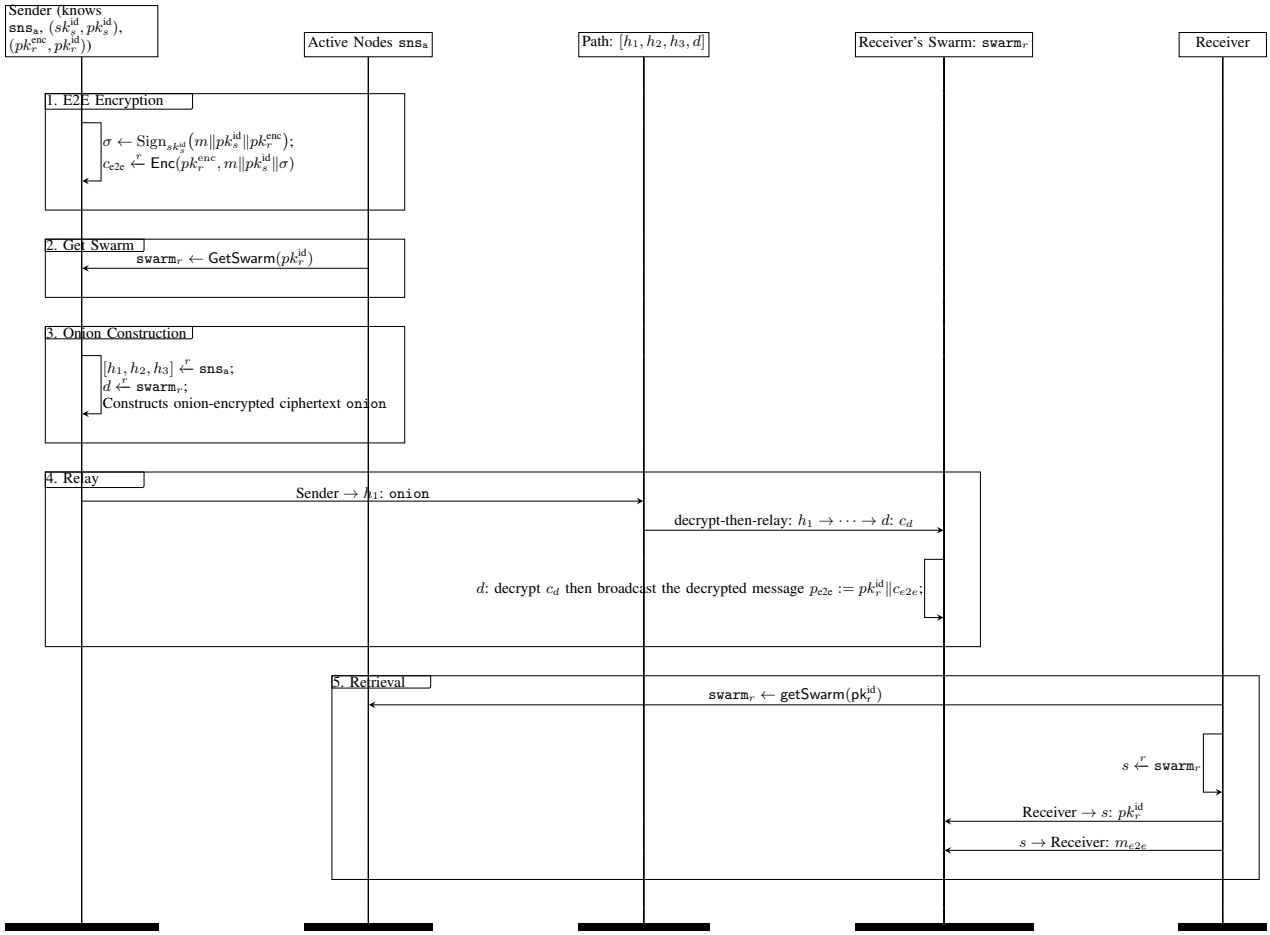


Figure 2: Session Send Message

Algorithm 3: $\mathsf{RunPulse}_\rho(S_i)$

Input: current state S_i
Output: Pulse block b
Operation:
for $r \leftarrow 0$ *to* 255 **do**
1 $PQ_r \leftarrow \mathsf{GenPulse}_\rho(S_i, r)$ ▷ Alg. 4
2 $b \leftarrow \mathsf{RunPulseRound}_\rho(PQ_r, S_i, r)$ ▷ Sec. 4
3 $\mathsf{UpdateLastPulse}(S_i.\mathtt{sns}, PQ_r, i)$ ▷ App. C.2
if b *is a valid block* **then**
4 **return** b
5 **return** b produced via PoW fallback

Algorithm 4: $\mathsf{GenPulse}_\rho(S_i, r)$

Context:
ρ: Network parameters
Input: Current state $S_i := \{B, \mathtt{sns}, \mathtt{txs}, Q\}$, Round r
Output: PQ : PQ
Operation:
1 $\mathtt{sns_a} \leftarrow [\mathtt{sn} \in S_i.\mathtt{sns} \mid \mathtt{sn.active}]$
2 $\mathtt{sns_a}.\mathtt{Sort}(\mathtt{key} = \mathtt{lastPulse})$
3 $E \leftarrow \mathsf{GetPulseEntropy}_\rho(S_i.\mathtt{B}, r)$ ▷ Alg. 5
4 $SI \leftarrow \mathsf{GetSI}_\rho(E, |\mathtt{sns_a}|)$ ▷ Alg. 6
5 $SN \leftarrow \mathsf{ApplySI}_\rho(SI, \mathtt{sns_a})$ ▷ Alg. 7
6 $p \leftarrow SN[0].pk$
7 $V \leftarrow \{SN[i].pk \mid 1 \leq i < \rho.n_{\mathrm{Pulse}}\}$
8 $PQ \leftarrow \{p, V\}$
9 **return** PQ

prefix of SN determines the new Pulse quorum: element $SN[0]$ is the block producer and nodes $SN[1 : n_{\mathrm{Pulse}} - 1]$ are validators.

Running Pulse quorum. Each Pulse round consists of multiple stages. For simplicity, we abstract it into four stages that are central to the vulnerabilities we uncovered:

1) **Handshake.** The network has finalised the nodes participating in Pulse for producing the next block. Now the validators broadcast a handshake message to other validators to indicate its activeness.
2) **Bitset.** Next, each validator produces a *bitset* V_i to indicate which validators are active based on the received handshake message (e.g., 00000000010 indicates that only the second validator is active).

Algorithm 5: $\mathsf{GetPulseEntropy}_\rho(B, r)$

Input: $B := [b_0, \ldots, b_i]$, round r
Operation:
$E \leftarrow []$
$\texttt{start} \leftarrow i - \rho.\texttt{pulseLag}$
$\texttt{end} \leftarrow \texttt{start} + \rho.n_{\text{Pulse}}$
for $j \leftarrow \texttt{start}$ **to** $\texttt{end} - 1$ **do**
  if b_j *is a Pulse block* **then**
    $e_j \leftarrow \texttt{Hash}(r \| b_j.\texttt{pulseRand})$
  else
    $e_j \leftarrow \texttt{Hash}(r \| b_j.\texttt{blockHash})$
  $E.\texttt{Append}(e_j)$
return E

Algorithm 6: $\mathsf{GetSI}_\rho(E, n)$

Operation:
$SI \leftarrow [0]$
$i_{\text{half}} \leftarrow \lfloor \frac{n}{2} \rfloor - 1$
for $i \leftarrow 1$ **to** $\rho.n_{Pulse} - 1$ **do**
  $i_{\max} \leftarrow \texttt{Min}(i_{\text{half}}, n - i)$
  $r \xleftarrow{\$(E[i])} \{0, \ldots, i_{\max} - 1\}$
  $i_{\text{swap}} \leftarrow i + r$
  $SI.\texttt{Append}(i_{\text{swap}})$
return SI

Validators then broadcast this bitset and sign received bitsets that they agree with. The quorum will agree on the bitset with the most validators' signatures V_i^*. All validators included in the final bitset V_i^* must participate in subsequent Pulse stages, and validators not included cannot participate in subsequent stages. Once a valid V_i^* is finalised, the block producer populates the block template with transactions in the mempool, then broadcasts it to the validators.

3) **Commit-Reveal.** Upon receiving the block template, each participating validator $v \in V_i^*$ generates a random value r_v, then broadcast the commitment $\mathsf{Hash}(r_v)$. Upon receiving commitments from all participating validators. The quorum proceeds to broadcast the preimage r_v. After receiving and verifying all revealed randomness $\{r_v\}_{v \in V_i^*}$. They are aggregated to form the final $\texttt{pulseRand}_i$ in the block header. This commit and reveal protocol means that a single honest validator serves to ensure a random $\texttt{pulseRand}_i$, up to the bias introducible by aborts.
4) **Sign.** Validators verify that the revealed randomness matches the commit. Upon verification, each validator computes an aggregated $\texttt{pulseRand}_i$, adds it to the block header and signs the block. This commit and reveal protocol ensures that the aggregated randomness $\texttt{pulseRand}_i$ remains random as long as one validator is honest.
Upon collecting $\geq k_{\text{P}}$ signatures on the final block, it is broadcasted to the network.

Algorithm 7: $\mathsf{ApplySI}_\rho(SI, SN)$

Operation:
for $i \leftarrow 0$ **to** $\rho.n_{Pulse} - 1$ **do**
  $x \leftarrow SN[SI[i]]$
  $SN[SI[i]] \leftarrow SN[i]$
  $SN[i] \leftarrow x$
return SN

3.4.2. Obligation quorum. Obligation quorums monitor service node activity and propose state changes. Each quorum has n_v validators and n_w workers. Validators test workers (uptime proofs, storage reachability, participation metrics), then propose state change (if any) for the node. Each state change requires $\geq k_{\text{O}}$ validator votes.

Quorum Generation. Immediately after each new block b is accepted, Oxen derives a new Obligation quorum and caches it to the state. $\mathsf{GenObl}_\rho(b, \texttt{sns})$ is called to generate a new Obligation quorum from the block's $\texttt{blockHash}$ and the current service node list $\texttt{sns}$. The algorithm extracts the list of active nodes $\texttt{sns}_\texttt{a}$ and decommissioned nodes $\texttt{sns}_\texttt{d}$, and sorts each list by public key value. The lists are then shuffled and combined to yield a new Obligation quorum.

Algorithm 8: $\mathsf{GenObl}_\rho(b, \texttt{sns})$

Output: OQ : $\texttt{OQ}$
Operation:
$\texttt{sns}_\texttt{a} \leftarrow [\texttt{sn} \in S_i.\texttt{sns} \mid \texttt{sn.active}]$
$\texttt{sns}_\texttt{d} \leftarrow [\texttt{sn} \in S_i.\texttt{sns} \mid \neg\texttt{sn.active}]$
$\texttt{sns}_\texttt{a}.\texttt{Sort}(\texttt{key} = \texttt{pk})$
$SN \leftarrow \mathsf{Shuffle}_\rho(b.\texttt{blockHash}, \texttt{sns}_\texttt{a}, \texttt{sns}_\texttt{d})$
$V \leftarrow \{SN[i].pk \mid 0 \leq i < \rho.n_v\}$
$W \leftarrow \{SN[i].pk \mid \rho.n_v \leq i < \rho.n_v + \rho.n_w\}$
$OQ \leftarrow \{V, W\}$
return OQ

Algorithm 9 describes the shuffling process, which seeds a pseudorandom generator with the block hash then applies a two-stage Fisher–Yates shuffle [44] to produce a shuffle of the two lists.

Algorithm 9: $\mathsf{Shuffle}(r, \texttt{sns}_\texttt{a}, \texttt{sns}_\texttt{d})$

Context:
$\pi_r(L)$: permutes list L, seeded by r
Output: shuffled list
Operation:
$Y \leftarrow |\texttt{sns}_\texttt{a}|;\ Z \leftarrow Y + |\texttt{sns}_\texttt{d}|;\ N \leftarrow \rho.n_v$
$L_0 \leftarrow [0, \ldots, Y - 1]$
$\pi_r(L_0)$
$L_1 \leftarrow L_0[N :] + [Y, \ldots, Z - 1]$
$\pi_r(L_1)$
return $[L_0[0 : N] \| L_1]$ mapped to $\texttt{sns}_\texttt{a} \| \texttt{sns}_\texttt{d}$

$$\text{ShuffledList} = [\overbrace{v_1, \ldots, v_{n_v}}^{\text{Validators}} \| \overbrace{w_1, \ldots, w_{n_w}}^{\text{Workers}} \| \overbrace{r_1, \ldots, r_{n_r}}^{\text{Rest of list}}]$$

The first $n_v + n_w$ entries of the shuffled list constitute the Obligation quorum, and their public keys are appended

to state.quorums.

Running Obligation quorum. Upon each block addition, $\mathsf{RunObl}_\rho(S_i)$ is called to run a cached Obligation quorum in the state S_i. The algorithm loads the cached quorum at height $h-\rho.\texttt{reorgBuffer}$. The validators perform a set of node performance checks on each worker w in the quorum, and calls $\mathsf{SelectStateChange}_\rho(w, \texttt{pass}, S_i)$ to check if a state change is required. If a worker receives $\geq k_\mathrm{O}$ votes proposing the same state change, validators collaboratively sign a state change transaction and broadcasts it for it to the network.

Algorithm 10: $\mathsf{RunObl}_\rho(S_i)$

Operation:

$h \leftarrow |S_i.\texttt{B}|$
$\texttt{txs} \leftarrow \emptyset$
if $h \leq \rho.\texttt{reorgBuffer}$ **then**
  return $\texttt{txs}$
$h_{\texttt{Obl}} \leftarrow h - \rho.\texttt{reorgBuffer}$
$q \leftarrow S_i.\texttt{Q}[h_{\texttt{Obl}}]$
foreach $w \in q.W$ **do**
  $\texttt{pass} \leftarrow$ service node check result of w
  $\texttt{sc} \leftarrow \mathsf{SelectStateChange}_\rho(w, \texttt{pass}, S_i)$ ▷ Alg. 12
  if $\texttt{sc} \neq \texttt{NOOP}$ **then**
    $\texttt{vote} \leftarrow$ a signed vote for $(h_{\texttt{Obl}}, w, \texttt{sc})$
    $\texttt{votes} \leftarrow \mathsf{OblVoting}_{\rho,q}(\texttt{vote})$ ▷ App. C.2
    if $|\texttt{votes}| \geq \rho.k_O$ **then**
      $\texttt{tx} \leftarrow \{h_{\texttt{Obl}}, \texttt{sc}, w, \texttt{votes}\}$
      $\texttt{txs}.\mathsf{Append}(tx)$
return $\texttt{txs}$

4. Oxen Vulnerabilities

This section presents four vulnerabilities in Oxen's quorum mechanisms discussed in Section 3.4. Each finding assumes that the adversary has already reached a privileged position to exploit the vulnerability, e.g., controlling one obligation quorum (having at least k_O attacker validators). We present each vulnerability and analyse the impact. Then, in Section 6, we show how the impact of individual flaw can be combined and amplified to influence over later quorums.

OV1: Decommissioning to control Pulse quorum

Key Idea. *An adversarially controlled Obligation quorum can selectively decommission nodes to control who is elected for the next Pulse quorum.*

Details. Let S_i be the state after block B_i, and let $\texttt{sns}_a^{(i)}$ denote the active node list derived from $S_i.\texttt{sns}$, sorted by lastPulse (Section 3.4.1). To generate the Pulse quorum at block height i and round r, the protocol first derives an entropy vector $E^{(i,r)}$ from a lagged window of previous blocks, then maps it to a list of indices:

$$SI^{(i,r)} \leftarrow \mathsf{GetSI}(E^{(i,r)}, |\texttt{sns}_\mathrm{a}^{(i)}|).$$

Observe that these steps only selects the indices, not yet the concrete service nodes occupying them. The actual Pulse quorum is only finalised when those indices are applied to the active node list:

$$SN^{(i,r)} \leftarrow \mathsf{ApplySI}(SI^{(i,r)}, \texttt{sns}_\mathrm{a}^{(i)}).$$

This separation yields a vulnerability. Because the entropy source is public and lagged, the index schedule $SI^{(i,r)}$ is predictable from the known chain state.

This gives adversarial Obligation quorums running between height $i-\texttt{pulseLag}$ to $i-1$ a privileged position to strategically decommission honest workers, thereby changing the composition of $\texttt{sns}_\mathrm{a}^{(i)}$. For instance, if the Obligation quorum at height $i-1$ is adversarial, the protocol defined in Algorithm 2 would proceed as follows:

Height i-1: $\texttt{txs}^{(i-1)} \leftarrow \mathsf{RunObl}_\rho(B_{i-1}, S_{i-1})$, where the adversarial quorum produces strategic decommission transactions of honest workers.

Height i-1: $B_i \leftarrow \mathsf{RunPulse}_\rho(S_{i-1})$, where the block producer adds $\texttt{txs}^{(i-1)}$ to the block.

Height i: $S_i.\texttt{sns} \leftarrow \mathsf{UpdateSNS}(S_{i-1}.\texttt{sns}, B_i)$, which successfully applies the decommission transactions and alters $\texttt{sns}_\mathrm{a}^{(i)}$

Height i: ...

Height i: $B_{i+1} \leftarrow \mathsf{RunPulse}_\rho(S_i)$, where pulse quorum at height i is finalised

These state changes alter $\texttt{sns}_\mathrm{a}^{(i)}$ before it is used to finalise the Pulse quorum at height i, thereby changing the list to which the indices $SI^{(i,r)}$ are applied to.

Practical exploitability. In Oxen, registering a service node requires possession of a certain amount of stake. Assume each node is registered using the minimum required stake, the fraction of service nodes controlled by an adversary equals the adversarial stake fraction. In the remainder of this paper, adversary fraction p refers to the fraction of service nodes controlled by the adversary.

To exploit OV1, the adversary must control an Obligation quorum. Let the number of obligation validators n_v be 10, and the threshold k_O be 7 (as on the mainnet), and adversarial fraction p, the probability that the next Obligation quorum is adversarial is approximated by: $P(W) = \sum_{k=7}^{10} \binom{10}{k} p^k (1-p)^{10-k}$. Assuming independent trials across blocks, the probability of the adversary winning within n blocks is $1-(1-P(W))^n$, which is 79.8% when $p = 0.3, n = 150$.

OV2: Biasing block randomness to control Obligation quorum

Key Idea. *An adversarially controlled Pulse quorum can choose the block to control who is elected for the next Obligation quorum.*

Details. Let V_i^* denote the participating-validator bitset finalised in the Bitset stage, b_i denote the block template. As described in Section 3.4.1, only validators included in final bitset V_i^* can participate in the subsequent Commit-Reveal, and Sign stages. For each participating validator v, its random share r_v is aggregated to form the Pulse randomness value $b_i.\texttt{pulseRand}$ for the block, and block hash $b_i.\texttt{blockHash}$ is derived from all header fields

of the block, including pulseRand. After b_i is accepted, Oxen derives the next Obligation quorum as

$$OQ_{i+1} \leftarrow \mathsf{GenObl}_\rho(b_i, S_{i+1}.\mathtt{sns}),$$

which then applies a deterministic shuffle to the sns seeded by b_i.blockHash to produce the quorum.

This yields the vulnerability. Suppose the adversary controls at least k_P validators in the current Pulse quorum. In the handshake stage, these validators can agree on a bitset V_i^* that excludes all honest validators, thereby preventing them from contributing random shares during the Commit-Reveal stage. Once the block template is fixed, every input to GenObl_ρ is public and deterministic except the random shares. Since only attacker-controlled validators can participate in the subsequent stages, the attacker can try different values for $\{r_v\}_{v \in V_i^*}$, compute the resulting pulseRand and block hash, evaluate the derived Obligation quorum OQ_{i+1} offline, and stop when OQ_{i+1} contains at least k_O attacker validators. The rest of the Pulse stages can then proceed honestly.

OV2 is not an attack on the obligation quorum generation algorithm, but rather, an attack on the input block hash. By controlling the bitset, a Pulse quorum with $\geq k_\mathrm{P}$ attacker validators can control the pulse randomness shares and hence the block hash, thereby biasing the next generated Obligation quorum in its favour.

Practical exploitability. The exploitability of OV2 can be approximated similarly as for OV1. Given $n_{\mathtt{Pulse}} = 12$ (11 validators), $k_\mathrm{P} = 7$, and adversarial fraction p, we have $P(W) = \sum_{k=7}^{11} \binom{11}{k} p^k (1-p)^{11-k}$. The probability of the adversary winning within n blocks is 96.2% when $p = 0.3, n = 150$.

OV3: Obligation state-change transaction constraints not enforced

OV3 arises from the lack of temporal binding to an Obligation quorum's authority. At block height h, Oxen intends to execute the cached quorum at height $h_{\mathtt{Obl}} = h - \rho.\mathtt{reorgBuffer}$ (Algorithm 10). Hence, an Obligation quorum produced at height i is only expected to vote once when the chain reaches height $i + \rho.\mathtt{reorgBuffer}$. However, the validation logic for state-change transactions, as shown in $\mathsf{ValidTx}_\rho(\mathtt{tx}, S)$, is insufficient to enforce this intended behaviour. It accepts a state-change transaction as long as: the referenced quorum exists (L3), the voter signatures and quorum membership are valid (L4-5), the worker index is valid (L6), and the referenced quorum height is within a generic vote-lifetime window (L7).

Crucially, it does not bind the authority of an Obligation quorum to its designated height, nor does it restrict the multiplicity of transactions produced by a quorum. This yields two vulnerabilities.

- **OV3.1: Early voting and late voting.** An Obligation quorum generated at block i is intended to send its state-change transactions only after a fixed delay (reorgBuffer), i.e., beginning at block height $i + \mathtt{reorgBuffer}$. However, the current implementation accepts any transaction issued by the most recently stored quorum, permitting an adversarial quorum to vote immediately upon election, creating an immediate effect on biasing the active node list, which increases the impact of OV1. Symmetrically, state-change transactions from a stale quorum (e.g., at height $i - \mathtt{reorgBuffer}$) may still be accepted by the network at arbitrary later heights.
- **OV3.2: Repeated voting.** A quorum may submit multiple distinct state-change transactions across different blocks. Since Oxen does not enforce a "one-vote-per-quorum" invariant, an attacker-dominant quorum can exert disproportionate influence by submitting repeated or contradictory votes.

Algorithm 11: $\mathsf{ValidTx}_\rho(\mathtt{tx}, S)$

Operation:

1 $OQ \leftarrow S.\mathtt{Q}[\mathtt{tx.height}]$
2 $h \leftarrow |S.\mathtt{B}|$
3 **if** $OQ = \mathbf{null}$ **then return** $\perp$
4 **if** $|\mathtt{tx.votes}| < \rho.k_O$ **then return** $\perp$
5 **if** $\neg\mathsf{ValidateVotes}(\mathtt{tx.votes}, S)$ ▷ App. C.2 **then return** $\perp$
6 $pk_w \leftarrow OQ.\mathtt{W}[\mathtt{tx}.i_w]$
7 $\mathtt{sn}_w \leftarrow \{\mathtt{sn} \in S.\mathtt{sns} \mid \mathtt{sn}.pk = pk_w\}[0]$
8 **if** $\mathtt{sn}_w = \mathbf{null}$ **then return** $\perp$
9 **if** $\neg(h - \rho.\mathtt{voteLife} \leq \mathtt{tx.height} \leq h)$ **then return** $\perp$
10 **if** tx.sc *is redundant* **then return** $\perp$
11 **return** $\top$

OV4: Skipping decommission step

In Oxen, service node state change follows a staged procedure, formalised by $\mathsf{SelectStateChange}_\rho$. An active service node that passes all tests undergoes no state change; a decommissioned node that passes all tests is *recommissioned*; a service node that failed test(s), will either be *decommissioned* and become temporarily inactive, or be *deregistered* and permanently removed from the state, depending on its remaining credit calculated based on its past activity/contribution.

Algorithm 12: $\mathsf{SelectStateChange}_\rho(w, \mathtt{pass}, S_i)$

Operation:

find $\mathtt{sn} \in S_i.\mathtt{sns}$ where $w.pk = \mathtt{sn}.pk$
if passed **then**
  if $\neg$sn.active **then**
    return RECOM
  return NOOP
else
  credit $\leftarrow$ Calculate credit based on past activity of sn
  if $\neg$sn.active **then**
    if credit ≥ 0 **then**
      return NOOP
    return DEREG
  else
    return DECOM

Oxen checks the validity of a proposed state change when validating the corresponding state-change transaction, as described in $\mathsf{ValidTx}_\rho(\mathtt{tx}, S)$. However, the existing check is too weak to enforce the intended procedure: it effectively only checks that the proposed transition is not redundant or impossible with respect to the node's current state. Crucially, Oxen does not enforce that each *deregistration* transaction of a node is preceded by a *decommission* transaction, or that a *decommissioned* node must not be *deregistered* if it still has remaining credit.

Therefore, an adversarial Obligation quorum can propose to *deregister* any worker regardless of its current state, and the resultant state-change will be accepted and applied by the network. This drastically increases the power of an adversarial Obligation quorum, as a *deregistered* node is permanently removed from the network.

4.1. Impact

Referring back to the implied security properties in Section 3.1, OV1 and OV2 undermine P4 (Quorum Selection Fairness), which is a core requirement for a PoS blockchain. By strategically decommissioning nodes (OV1) or biasing block randomness (OV2), an attacker can skew both Pulse and Obligation quorum formation. This gives the adversary-controlled nodes a disproportionate advantage in future elections. OV3 breaks P3 (Temporal Quorum Authority), and OV4 breaks P5 (State Change Integrity), further amplify the power of an adversary by granting excessive power to the Obligation quorum.

As discussed in Section 2.1.2, Byzantine fault tolerant systems can tolerate at most one-third of adversarial participants, equivalently, one-third adversarial stake in PoS systems such as Oxen. In the next section, we show via simulation that combining these vulnerabilities can lead to practical network takeover without exceeding this threshold; our simulations are somewhat conservative because of the greedy approach they take to exploiting the vulnerabilities, it is likely they underestimate the power of a sophisticated adversary.

If an adversary had exploited these vulnerabilities for network takeover, this would have caused the honest nodes to be decommissioned. It seems a safe assumption that the honest node maintainers would then have complained and the attack would have been noticed. There are various more subtle exploitations of these vulnerabilities which do not result in network takeover but would advantage the adversary in other ways; we have not carefully examined the block history of Oxen for such attacks but it is clear from our discussion with the developers that they do not believe exploitation is occurring or has occurred.

5. Session Vulnerabilities

Because Session runs atop the same service node infrastructure as Oxen, consensus–layer vulnerabilities (Section 4) can directly undermine Session's anonymity. In this section, we focus on weaknesses specific to Session's protocols, independent of Oxen. When we began our examination of Session, it was using version one of its group chat protocol. It has since migrated to group protocol version two and discontinued version one; this change occurred in November 2024. We included in Appendix B vulnerabilities which apply only to version one.

5.1. SV1. Service Node Discovery

As discussed, Session relies on a network of service nodes to relay messages and host group data. To join the network, a client first needs an entry point that provides the current set of active service nodes. Session bootstraps this process using a small set (currently three) of hardcoded seed nodes. On first use, the client queries three randomly chosen seeds, each of which returns its snapshot of the active service node list. The client accepts the result if the three snapshots are consistent. After bootstrapping, the client refreshes its view by querying random service nodes from the locally stored list. In theory, this approach should harden the protocol against a corrupt single seed node allowing the network to be spoofed to the client; in reality, the implementation introduces additional single points of failure rather than distributing trust.

A malicious seed node can supply an adversarial list, making all future onion routes through attacker-controlled nodes. While the client would query three random seed nodes then compare the received list to check consistency, Session only has a small pool of hardcoded seed nodes (currently three), meaning that there is a $1/27$ probability of querying the same seed three times. Thus, control of a single seed suffices to subvert a client's view of the network with fairly high probability.

5.1.1. Current Group Chat Protocol (version two).

Both version one issues (Appendix B) are addressed in version two: all state changes (including `add`) are admin–only, and member departure triggers an admin–initiated rekey. We nevertheless observed a *weak binding of control messages*: group actions are split into:

1) a *configuration update* that the admin publishes to the swarm (encrypted under the group key) and that clients periodically poll; and
2) a *control message* used for immediate UI feedback, e.g., the control message for `add` (invite) is:

$$m = \big(\mathtt{sender}, t, \mathtt{inner}\big)$$
$$\mathtt{inner} = \big(\mathtt{op}, \overbrace{\langle pk_1, \ldots, pk_\ell \rangle}^{\text{invitees}}, \ldots, \sigma\big)$$

For control messages, the signature (σ) covers only *type* (`op`) and *timestamp* (t) and omits the content (invitees, in this example). Consequently, an adversary can record and replay a valid admin-signed control message, substitute the target while reusing the same signature and timestamp. Honest members accept the forged message and display it as a part of the group transcript, but the impact is limited to a misleading UI notification (e.g., "x was invited to join the group"), as the authoritative group state is derived from admin–published metadata in the configuration plane. Figure 5 shows a `setName` replay where the attacker substitutes a different name under the same signature/timestamp (corresponding logs in Appendix B.1.3).

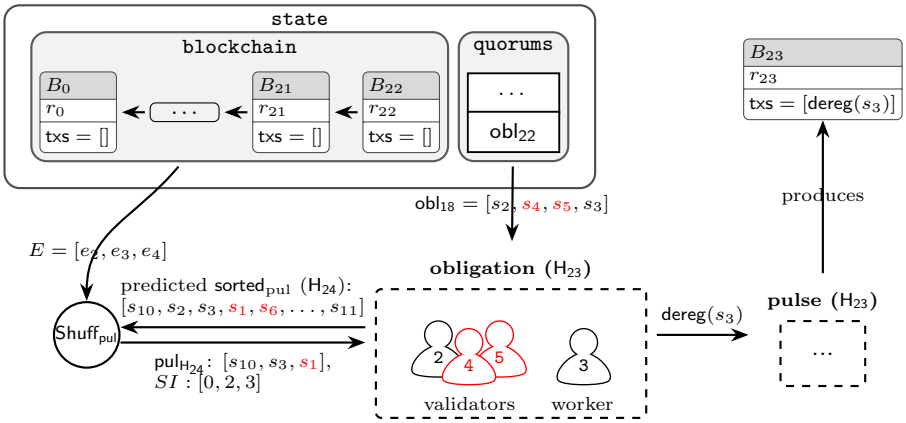


Figure 3: OV1: Obligation quorum biasing Pulse quorum at height 23

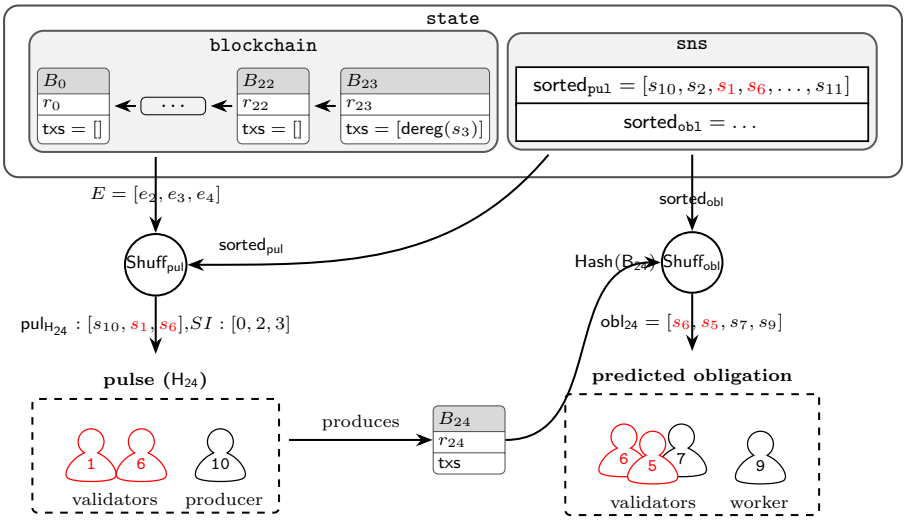


Figure 4: OV2: Pulse quorum biasing Obligation quorum at height 24

6. Network Takeover Attack on Oxen

Combining vulnerabilities discussed in the previous section, we present a practically-exploitable network takeover attack that allows an adversary controlling $\leq$ 30% of the stake to gain control over the entire network within days. Then, we analyse its impact with simulations.

As mentioned in Section 3.4, the selection algorithm of Pulse maps block entropy to an index in the active node list, while state-change transactions produced by an Obligation quorum can affect the construction of the active node list. Therefore, an attacker-controlled Obligation quorum can strategically decommission the test nodes such that more attacker-controlled nodes will fall into indices for a future Pulse quorum (OV1).

Additionally, each Obligation quorum is generated from the hash of a single block, meaning that once attackers control a Pulse quorum, producing a block that generates an attacker-controlled Obligation quorum is straightforward (OV2). The `reorgBuffer` mechanism limits the immediate impact of OV2 by preventing a freshly generated Obligation quorum from voting immediately. However, the inadequate vote validation (OV3) allows this to be bypassed.

6.1. Demonstrative Example

To illustrate the network takeover attack with a concrete example, we consider a simplified protocol model:

$n_{\textbf{Pulse}} = 3$**:** the Pulse quorum consists of two validators and one block producer, requiring 2/2 validator signatures. $n_v = 3$, $n_w = 1$: the Obligation quorum consists of three validators and one worker, requiring 2/3 validator signatures and `reorgBuffer` = 5.

B_i each block contains a Pulse randomness value r and a set of transactions txs.

$\mathsf{Shuff}_{\mathsf{pul}}(E, SN)$ combines Algorithm 6 with Algorithm 7, consistent with the Pulse quorum selection protocol described in Section 3.4.1.

$\mathsf{Shuff}_{\mathsf{obl}}(r, \mathsf{sns}_{\mathsf{a}})$ follows Algorithm 9, except that the decommissioned node list argument $\mathsf{sns}_{\mathsf{d}}$ is omitted, as we assume no pending decommission transactions and that an attacker-controlled Obligation quorum exploits OV4 to directly deregister nodes, leaving $\mathsf{sns}_{\mathsf{d}}$ empty throughout the example.

`state` We assume that the network is synchronous and hence the state object is the same for all nodes. The sns stores the active node list, $\mathsf{sort}_{\mathsf{pul}}$ returns the list sorted by lastPulse and $\mathsf{sort}_{\mathsf{obl}}$ returns the list sorted

by public key value.

At block height 23 (Figure 3), the running Obligation quorum is retrieved from cache as `state.quorums`$[23 - \texttt{reorgBuffer}] = \mathsf{obl}_{18}$. This quorum is controlled by adversarial validators s_4 and s_5, with an honest worker s_3. To exploit OV1 and bias the next Pulse quorum at height 24, the adversarial validators predict the next quorum formation using entropy values from blocks B_2–B_4 and the predicted active node list $\mathsf{sns}_{\mathsf{active}}$, sorted by `lastPulse` as it would appear at height 24 (which can be done by updating the `lastPulse` field of the current Pulse participants to 23, then re-sorting the list to simulate the view at the next height). The attackers then call $\mathsf{Shuff}_{\mathsf{pul}}$ to yield the predicted Pulse quorum s_{10}, s_3, s_1 and swap indices $[0, 2, 3]$. Although only s_1 is adversarial in the predicted Pulse quorum, the attackers observe that removing the current worker s_3 (at index 2 of the sorted active node list) would shift s_1 and s_6 to indices 2 and 3, respectively, making the next Pulse quorum attacker-controlled. They proceed to submit a deregistration transaction $\mathsf{dereg}(s_3)$ to the mempool, such that the current Pulse quorum can include it to the new block B_{23} they are producing.

At height 24 (Figure 4) the deregistration has taken effect and s_3 is absent from $\mathsf{sns}_{\mathsf{active}}$. With the same swap indices the actual Pulse becomes $[s_{10}, s_1, s_6]$, and the attacker-controlled Pulse begins executing and exploits OV2 to extend control: by varying the Pulse randomness they influence the block hash and hence the Obligation quorum derived from that block. Concretely, the adversary repetitively calls $\mathsf{Shuff}_{\mathsf{obl}}(\mathsf{Hash}(B_{24}), \mathsf{sort}_{\mathsf{obl}})$ over different values for B_{24} with different randomness r_{24} until the derived Obligation quorum obl_{25} contains a winning attacker majority (e.g., s_6, s_5).

While the running Obligation quorum at height 25 should be obl_{20} given the cache delay value, the attacker-controlled quorum obl_{25} can exploit OV3 to immediately start running. Repeating this process yields a chaining effect in which biased Pulses and biased Obligations reinforce one another.

6.2. Analysis

6.2.1. Oxen Simulation. We used python scripts to simulate the Oxen mainnet with the following parameters, matching the corresponding parameters used by the mainnet (see Appendix C.3 for mainnet configuration):

- network size: $n = 2000$, similar to the state of mainnet (≈ 1600 nodes)
- Pulse quorum: $n_{\text{Pulse}} = 12$, requiring $7/11$ validator signatures
- Obligation quorum: $n_v = 10, n_w = 50$, requiring $7/10$ validator signatures

We simulate an attacker controlling $p \cdot n$ nodes. Oxen does not state a formal global Byzantine threshold. However, one of its design documents considers adversary fractions 0.25, 0.33, and 0.5. We therefore do not claim a precise adversarial fraction that Oxen should tolerate. Instead, we evaluate our attacks empirically across a range of adversarial fractions, and treat 0.33 as a high-end benchmark. Appendix D provides further details on the simulation.

We define the adversary's winning condition as controlling $\geq 80\%$ of the active nodes. Table 2 records the number of blocks required for the adversary to win under different combinations of vulnerabilities. p denotes the attacker fraction, and each subsequent column shows the median number of blocks required for the adversary to win (more data in App. D.2).

TABLE 2: Simulated Oxen Attack Results

p	OV1,2	OV1,2,3.1	OV1,2,3.2	p	OV1,2,4
0.26	3300	2600	1100	0.21	3700
0.27	1900	2100	800	0.23	2700
0.29	1400	1300	700	0.25	1600
0.31	1200	1100	500	0.27	1000
0.33	1000	900	400	0.29	900

From the simulation results, we observe that when the attacker exploits only OV1 and OV2, the probability of takeover is negligible for attacker fractions $p \leq 0.25$, as the adversary does not succeed within 10000 simulated blocks. However, once $p \geq 0.26$, the adversary consistently succeeds in taking over the network.

When OV3.2 is also exploited, the takeover occurs significantly faster due to the ability of past adversarial Obligation quorums to reissue votes. Given that the target block time of the Oxen mainnet is 2 minutes (approximately 720 blocks per day), our simulation suggests that an adversary controlling ≥ 0.26 of service nodes could take control of the network within 2 days.

OV4 (deregistration without decommission) further reduces the stake required for successful network takeover, as adversarial Obligation quorums can trivially remove honest nodes from the network permanently.

6.2.2. Oxen Emulation. We validated the practical exploitability of each identified vulnerability and their interaction in a 40-node local Oxen testnet using a lightly patched `oxen-core` [1] daemon. We modified Oxen's `local-testnet` scripts and core consensus components (Pulse, Obligation quorum, service node list classes) to embed attacker logic into the daemon. While details of the emulation setup are omitted for ethical reasons, the artefact in Appendix A includes supporting logs of the runs and accompanying documentation.

7. Mitigations

In this section, we will discuss mitigations to the vulnerabilities we discovered; we will omit discussion of mitigations of the admin impersonation and rekeying vulnerabilities in version one of the Session group chat protocol since these have now been fixed.

7.1. Session Mitigations

SV1 This vulnerability could be partially mitigated by increasing the number of hard-coded seed nodes, and ensuring the client queries different nodes. In the longer term, the bootstrapping process could be tied to the blockchain state rather than a static seed list.

SV2 and SV3 Sign the entire control message (operation type, timestamp, and message target/content).

1. https://github.com/oxen-io/oxen-core

7.2. Oxen Mitigations

OV1 This vulnerability could be mitigated by selecting parties rather than indices; this would remove the Obligation quorum's impact yet would increase the number of failed Pulse rounds, as some service nodes selected for a quorum might no longer be active when the quorum is meant to run. An alternative would be to delay the effect of the Obligation quorums: a state-change transaction cannot be added to a block until it has remained in the mempool for a certain number of blocks so that once the indices for a round are known, there is no ability to change the active node list to exploit this information.

OV2 The ability of the Pulse quorum to bias the other quorums seems inherent to this style of Proof of Stake blockchain. However, patching OV3 and OV4 will significantly mitigate the ability of a dishonest Obligation quorum to affect the system. Consequently, even though the adversary retains any advantage in being elected on this quorum, it would derive less benefit from being elected.

OV3 The transaction validation checks should be strengthened. To mitigate OV3, we suggest enforcing a more restricted voting window than the current generic vote lifetime window (h denotes the current block height): $h - \texttt{voteLife} \leq \texttt{tx.height} \leq h$. Concretely, $\mathsf{ValidTx}_\rho(\texttt{tx}, S)$ (L9) uses $h - \texttt{reorgBuffer} - \Delta \leq \texttt{tx.height} \leq h - \texttt{reorgBuffer}$ as the guarding condition, where Δ is 0 or a small value that accounts for network delay.

OV4 $\mathsf{ValidTx}_\rho$ should verify that the state-change is legal given the node's current state. In particular, it should check that a node is *decommissioned* before it can be *deregistered*. Similar to Algorithm 12, the function should compute the node's remaining credit to determine whether the node is within its grace period.

Implementation Status. The Oxen developers have been patching the protocol following our disclosure. Several countermeasures are scheduled for inclusion in the next network version upgrade (hard fork 23). The main changes include revised network parameters: a larger Pulse quorum size ($n_{\text{pulse}} = 25$) and an increased signature threshold requiring $16/24$ validator signatures per Pulse block, partially mitigating OV1. In addition, new validation checks for state-change transactions have been added to ensure that decommission and grace periods cannot be bypassed, addressing OV4.

We reran the network-takeover simulations with the updated parameters and OV4 exploitation disabled. The results show that an adversary now needs roughly $p \geq 0.31$ of the network to succeed.

TABLE 3: Simulated Oxen Attack Results (With Patch)

p	OV1,2,3.2
0.30	Failed
0.31	2100
0.32	1200
0.33	1100

This improvement likely comes from two factors:

- Patching OV4 prevents attacker-controlled Obligation quorums from producing permanent state-change effects. In the original design, an adversarial Obligation quorum could permanently remove honest service nodes directly, allowing the adversary fraction to increase monotonically. After the patch, Obligation quorums can only temporarily decommission nodes. This significantly increases the difficulty of network takeover for adversaries controlling smaller fractions of the network, as honest quorums can later issue recommission transactions to restore decommissioned workers, which allows the system to recover from adversarial decisions.
- Increasing the Pulse quorum size and raising its signature threshold reduces the probability of satisfying the precondition required to exploit OV2, which is to have an adversarial Pulse quorum ($\geq k_{\text{P}}$ attacker validators). Under the new parameters, Pulse quorums require signatures from $16/24 \approx 66.7\%$ of validators, compared to $7/11 \approx 63.7\%$ previously. This raises the adversarial fraction required for a quorum takeover and reduces the likelihood that a randomly selected quorum is adversarial. The larger Pulse quorum threshold also weakens the effectiveness of the OV1 attack, where an adversarial Obligation quorum attempts to decommission a set of honest workers in order to bias the composition of future Pulse quorums. With the increased Pulse quorum size and signature threshold, the same pool of ≤ 50 Obligation workers must now satisfy the larger validator threshold (from 7 to 16). In our simulations, this significantly reduces the success rate of the heuristic search used by the OV1 adversary.

We note that $p = 0.31$ is still not optimal because:

- our simulated OV1 adversary is greedy, though the optimal adversary is likely not much better; and
- 0.31 remains below the theoretical Byzantine tolerance upper-bound implied by the quorum parameters: $(16/24) \times 2 - 1 \approx 0.333$, and the simulated takeover completes within 2100 blocks (< 3 days), suggesting that while the patch improves resilience, Oxen can still be compromised by sub-Byzantine adversaries over a short period.

8. Conclusion

This paper presents the first protocol-level analysis of Session and its Oxen backbone, revealing seven vulnerabilities spanning client protocol (Sec. 5) and Oxen consensus (Sec. 4). In particular, we showed how biasable quorums and insufficient vote validation can turn into practical network-takeover attacks with sub-majority adversarial stake, undermining the integrity guarantees on which Session's anonymity depends. We responsibly disclosed all issues and outlined mitigations. We hope this work facilitates improving the security of Session and similar systems.

Acknowledgment

Thomas Haines is the recipient of an Australian Research Council Australian Discovery Early Career Award (project number DE220100595).

References

[1] M. Marlinspike and T. Perrin, "The x3dh key agreement protocol," *Open Whisper Systems*, vol. 283, 2016.

[2] T. Perrin and M. Marlinspike, "The double ratchet algorithm," https://signal.org/docs/specifications/doubleratchet/, 2016.

[3] M. Marlinspike and T. Perrin, "The sesame algorithm: session management for asynchronous message encryption," *Revision*, vol. 2, pp. 2017–04, 2017.

[4] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the signal messaging protocol," in *2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, April 26-28, 2017*. IEEE, 2017, pp. 451–466. [Online]. Available: https://doi.org/10.1109/EuroSP.2017.27

[5] J. Alwen, S. Coretti, and Y. Dodis, "The double ratchet: Security notions, proofs, and modularization for the signal protocol," *IACR Cryptol. ePrint Arch.*, p. 1037, 2018. [Online]. Available: https://eprint.iacr.org/2018/1037

[6] J. Brendel, M. Fischlin, F. Günther, C. Janson, and D. Stebila, "Towards post-quantum security for signal's X3DH handshake," in *Selected Areas in Cryptography - SAC 2020 - 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21-23, 2020, Revised Selected Papers*, ser. Lecture Notes in Computer Science, O. Dunkelman, M. J. J. Jr., and C. O'Flynn, Eds., vol. 12804. Springer, 2020, pp. 404–430. [Online]. Available: https://doi.org/10.1007/978-3-030-81652-0_16

[7] G. Greenwald, "Nsa collecting phone records of millions of verizon customers daily," *The Guardian*, 2013. [Online]. Available: https://www.theguardian.com/world/2013/jun/06/nsa-phone-records-verizon-court-order

[8] A. Belanger, "Apple admits to secretly giving governments push notification data," https://arstechnica.com/tech-policy/2023/12/apple-admits-to-secretly-giving-governments-push-notification-data, December 2023.

[9] Session Technology Foundation, "Session: Private messenger," https://getsession.org/, 2020, end-to-end encrypted messenger using onion routing over a decentralised node network; developed since 2018 by the Oxen Privacy Tech Foundation (OPTF), publicly released in 2020.

[10] A. Linton, "Connecting one million users," https://getsession.org/blog/connecting-one-million-users, August 2024.

[11] Quarkslab, "Oxen session audit technical report," https://blog.quarkslab.com/resources/2021-05-04_audit-of-session-secure-messaging-application/20-08-Oxen-REP-v1.4.pdf, May 2021.

[12] M. G. Reed, P. F. Syverson, and D. M. Goldschlag, "Anonymous connections and onion routing," *IEEE J. Sel. Areas Commun.*, vol. 16, no. 4, pp. 482–494, 1998. [Online]. Available: https://doi.org/10.1109/49.668972

[13] R. Dingledine, N. Mathewson, and P. F. Syverson, "Tor: The second-generation onion router," in *Proceedings of the 13th USENIX Security Symposium, August 9-13, 2004, San Diego, CA, USA*, M. Blaze, Ed. USENIX, 2004, pp. 303–320. [Online]. Available: http://www.usenix.org/publications/library/proceedings/sec04/tech/dingledine.html

[14] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Cryptography Mailing list at https://metzdowd.com*, 03 2009.

[15] J. A. Garay, A. Kiayias, and N. Leonardos, "The bitcoin backbone protocol: Analysis and applications," in *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II*, ser. Lecture Notes in Computer Science, E. Oswald and M. Fischlin, Eds., vol. 9057. Springer, 2015, pp. 281–310. [Online]. Available: https://doi.org/10.1007/978-3-662-46803-6_10

[16] L. Lamport, R. E. Shostak, and M. C. Pease, "The byzantine generals problem," *ACM Trans. Program. Lang. Syst.*, vol. 4, no. 3, pp. 382–401, 1982. [Online]. Available: https://doi.org/10.1145/357172.357176

[17] M. Castro and B. Liskov, "Practical byzantine fault tolerance," in *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI), New Orleans, Louisiana, USA, February 22-25, 1999*, M. I. Seltzer and P. J. Leach, Eds. USENIX Association, 1999, pp. 173–186. [Online]. Available: https://dl.acm.org/citation.cfm?id=296824

[18] QuantumMechanic, "Proof of stake instead of proof of work," https://bitcointalk.org/index.php?topic=27787.0, Jul. 2011, bitcoin Forum, Development & Technical Discussion. Widely regarded as the original public proposal of the proof-of-stake consensus mechanism; posted July 11, 2011.

[19] S. King and S. Nadal, "Ppcoin: Peer-to-peer crypto-currency with proof-of-stake," https://decred.org/research/king2012.pdf, 2012.

[20] I. Bentov, A. Gabizon, and A. Mizrahi, "Cryptocurrencies without proof of work," in *Financial Cryptography and Data Security - FC 2016 International Workshops, BITCOIN, VOTING, and WAHC, Christ Church, Barbados, February 26, 2016, Revised Selected Papers*, ser. Lecture Notes in Computer Science, J. Clark, S. Meiklejohn, P. Y. A. Ryan, D. S. Wallach, M. Brenner, and K. Rohloff, Eds., vol. 9604. Springer, 2016, pp. 142–157. [Online]. Available: https://doi.org/10.1007/978-3-662-53357-4_10

[21] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A provably secure proof-of-stake blockchain protocol," in *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part I*, ser. Lecture Notes in Computer Science, J. Katz and H. Shacham, Eds., vol. 10401. Springer, 2017, pp. 357–388. [Online]. Available: https://doi.org/10.1007/978-3-319-63688-7_12

[22] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling byzantine agreements for cryptocurrencies," in *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*. ACM, 2017, pp. 51–68. [Online]. Available: https://doi.org/10.1145/3132747.3132757

[23] B. David, P. Gazi, A. Kiayias, and A. Russell, "Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain," in *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part II*, ser. Lecture Notes in Computer Science, J. B. Nielsen and V. Rijmen, Eds., vol. 10821. Springer, 2018, pp. 66–98. [Online]. Available: https://doi.org/10.1007/978-3-319-78375-8_3

[24] S. Micali, M. O. Rabin, and S. P. Vadhan, "Verifiable random functions," in *40th Annual Symposium on Foundations of Computer Science, FOCS 1999, New York, NY, USA, October 17-18, 1999*. IEEE Computer Society, 1999, pp. 120–130. [Online]. Available: https://doi.org/10.1109/SFFCS.1999.814584

[25] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the signal messaging protocol," in *2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, April 26-28, 2017*. IEEE, 2017, pp. 451–466. [Online]. Available: https://doi.org/10.1109/EuroSP.2017.27

[26] N. Kobeissi, K. Bhargavan, and B. Blanchet, "Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach," in *2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, April 26-28, 2017*. IEEE, 2017, pp. 435–450. [Online]. Available: https://doi.org/10.1109/EuroSP.2017.38

[27] D. Van Dam, "Analysing the signal protocol," Master's Thesis, Radboud University Nijmegen, 2019. [Online]. Available: https://www.ru.nl/publish/pages/769526/z00b_2019_thesis_dion_van_dam_2019_eerder.pdf

[28] K. G. Paterson, M. Scarlata, and K. T. Truong, "Three lessons from threema: Analysis of a secure messenger," in *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*, J. A. Calandrino and C. Troncoso, Eds. USENIX Association, 2023, pp. 1289–1306. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/paterson

[29] Bridgefy, Inc., "Bridgefy: Offline mesh messaging app," https://bridgefy.me/, bluetooth-based mesh networking app enabling decentralised, internet-free messaging.

[30] PrivaTegrity Corporation, "xx Messenger," https://learn.xx.network/dapps/xxm/, 2022, decentralised encrypted instant messaging application using the cMix mixnet protocol on the xx network, developed by David Chaum; public release January 25, 2022.

[31] D. Chaum, D. Das, A. Kate, F. Javani, A. Krasnova, A. T. Sherman, and J. de Ruiter, "cmix: Anonymization by high-performance scalable mixing," Cryptology ePrint Archive, Tech. Rep., 2016.

[32] D. Chaum, D. Das, F. Javani, A. Kate, A. Krasnova, J. de Ruiter, and A. T. Sherman, "cmix: Mixing with minimal real-time asymmetric cryptographic operations," in *Applied Cryptography and Network Security - 15th International Conference, ACNS 2017, Kanazawa, Japan, July 10-12, 2017, Proceedings*, ser. Lecture Notes in Computer Science, D. Gollmann, A. Miyaji, and H. Kikuchi, Eds., vol. 10355. Springer, 2017, pp. 557–578. [Online]. Available: https://doi.org/10.1007/978-3-319-61204-1_28

[33] Briar Project, "Briar: Secure messaging, anywhere," https://briarproject.org/, 2018, peer-to-peer encrypted messaging application using the Tor network for onion routing, with fallback to Bluetooth and Wi-Fi; first public release on Google Play, April 2018.

[34] J. Brooks, "Ricochet: Anonymous peer-to-peer instant messaging," https://github.com/ricochet-im/ricochet, 2014, peer-to-peer instant messaging client built on Tor hidden services, providing metadata-free anonymous communication; originally released as Torsion IM, renamed Ricochet in June 2014.

[35] K. Jefferys, S. Harman, J. Ross, and P. McLean, "Loki: Private transactions, decentralised communication," Whitepaper, Version 3, July 2018, available at https://loki.network.

[36] Y. Song, "Cryptography in the wild: Briar," 2023, master thesis (available at https://ethz.ch/content/dam/ethz/special-interest/infk/inst-infsec/appliedcrypto/education/theses/report_YuanmingSong.pdf).

[37] M. R. Albrecht, J. Blasco, R. B. Jensen, and L. Mareková, "Mesh messaging in large-scale protests: Breaking bridgefy," in *Topics in Cryptology - CT-RSA 2021 - Cryptographers' Track at the RSA Conference 2021, Virtual Event, May 17-20, 2021, Proceedings*, ser. Lecture Notes in Computer Science, K. G. Paterson, Ed., vol. 12704. Springer, 2021, pp. 375–398. [Online]. Available: https://doi.org/10.1007/978-3-030-75539-3_16

[38] M. Williams, "Secure messaging apps comparison," https://www.securemessagingapps.com/, Dec. 2025, gitHub: https://github.com/markxwilliams/securemess.

[39] Cyble Editorial, "Top 10 secure messaging apps for encrypted chats," https://cyble.com/knowledge-hub/top-secure-messaging-apps-encrypted-chats/, Mar. 2026.

[40] Express VPN, "Most secure messaging apps in 2026: Ranked and reviewed," https://www.expressvpn.com/blog/best-messaging-apps/, Aug. 2025.

[41] PCMag Australia, "The best private messaging apps for 2025," https://au.pcmag.com/security/94184/the-best-secure-messaging-apps-for-2022, Sep. 2025.

[42] Session Technology Foundation, "Session litepaper: Send messages, not metadata," Session Private Messenger, 2020, accessed: 2026-03-26. [Online]. Available: https://getsession.org/assets/papers/Litepaper_Session_private_messenger.pdf

[43] K. Jefferys, M. Shishmarev, and S. Harman, "Session: End-to-end encrypted conversations with minimal metadata leakage," February 2020, originally submitted 11 Feb 2020; revised version (v3) 2 Jul 2024. [Online]. Available: https://arxiv.org/abs/2002.04609

[44] R. Durstenfeld, "Algorithm 235: Random permutation," *Commun. ACM*, vol. 7, no. 7, p. 420, 1964. [Online]. Available: https://doi.org/10.1145/364520.364540

Appendix A. Data Availability

Released data. See `README.md` inside the artefact for more details.

- **Simulation code:** Python scripts used to demonstrate the network takeover attacks.
- **Emulation setup script:** The python scripts for spinning up a local Oxen testnet.
- **Emulation log:** The log output for emulated attacks

We omit the patched Oxen daemon source code that implements the live attack algorithm (i.e. the modified `pulse.cpp` and `service_node_quorum_cop.cpp`), as publishing those patches prior to upstream fixes could enable real-world exploitation against the Oxen mainnet.

Access. The artefact is available at https://github.com/pogen300/oxen-blockchain-research/

Appendix B. Additional Session Vulnerabilities

We will now present two vulnerabilities in group chat version one, which have subsequently become defunct with the release of group chat version two.

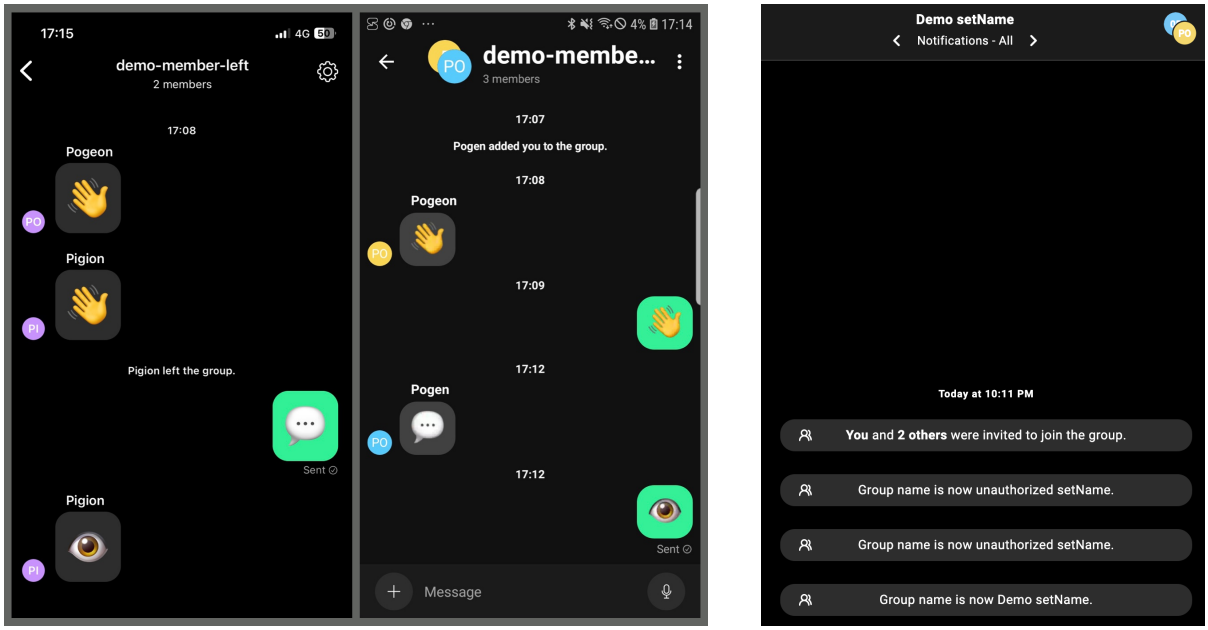
B.1. Group Chat Protocol

In both protocol versions, the group creator is the sole administrator in practice (the protocol permits multiple admins, but the client provides no mechanism to add or transfer admin status). Control messages include `promote`, `add`, `remove`, `leave`, and `rekey`.

B.1.1. SV2. Insufficient rekeying mechanism in Legacy Group Chat Protocol (version one). Voluntary departure does not trigger a rekey in version one. A leaving member sends a `leave` command and unsubscribes from swarm updates but retains valid keys and can continue to decrypt and inject messages until a later rekey (e.g., triggered by a `remove`) propagates. Even after `rekey`, honest clients may fetch old keys from storage to decrypt injected messages under stale keys (Figure 5).

B.1.2. SV3. Admin impersonation in Legacy Group Chat Protocol (version one). In group chat protocol version one, any member can invite its contacts to the group via an `add` command that embeds the group state, including the admin list. Any member could craft an invitation `(groupID, name, members, admins, timestamp)` that adds itself as an admin before forwarding to a new user, thereby gaining admin privileges over the invitee and over users invited by the invitee thereafter.

Figure 6 demonstrates admin impersonation: The honest admin is A (left), but a malicious member B (middle) invites a new member C (right) while listing itself as the admin. From this point onward, the new member C believes B is the admin. When B issues a remove operation against A (the real admin), C's group view is updated to reflect only 2 members. Meanwhile, the A's view still contains 3 members, as it correctly ignores the unauthorised removal.



(a) Insufficient rekeying

(b) `setName` command replay

Figure 5: Group chat attack demo

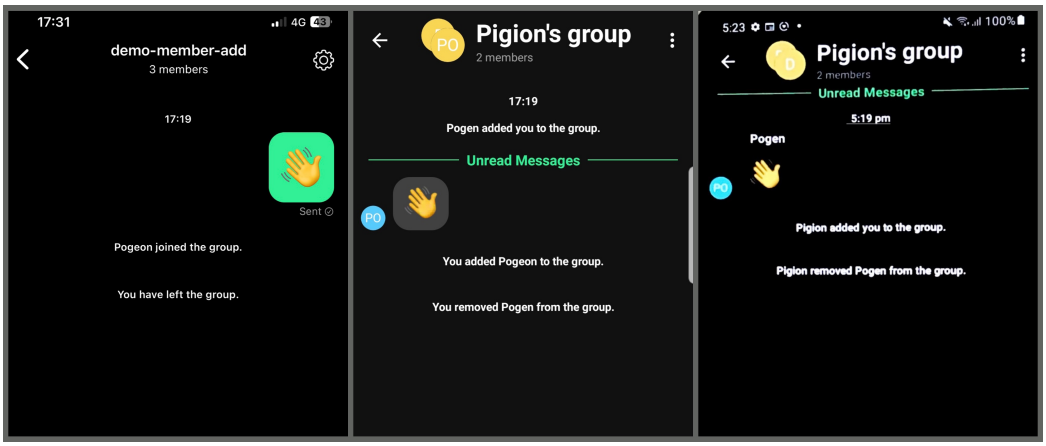


Figure 6: Demo of admin impersonation from the admin (Left), attacker (Middle), and new member's (Right) views

B.1.3. `setName` Replay Log. The first message is a legitimate group name update message from the admin, observed by the attacker. The second message is the attacker's forged message, reusing the signature and timestamp, but a different `updatedName`.

```
2025-08-25 22:12:04.580 22381-22405 ... D infoChangeMessage {
 type: NAME
 updatedName: "Demo setName"
 adminSignature: "\306k\027\357..."
}
2025-08-25 22:12:04.581 22381-22405 ... D signature: c66b17efd...
2025-08-25 22:12:04.581 22381-22405 ... D timestamp: 1756123921633

2025-08-25 22:12:59.973 22533-22579 ... D infoChangeMessage {
 type: NAME
 updatedName: "unauthorized setName"
 adminSignature: "\306k\027\357..."
}
2025-08-25 22:12:59.973 22533-22579 ... D signature: c66b17ef...
2025-08-25 22:12:59.973 22533-22579 ... D timestamp: 1756123921633
```

Appendix C. Oxen Protocol

C.1. Abstraction of Oxen Protocols

Our model of Oxen does not reproduce the byte-level implementation of `oxen-core`, but preserves the core protocol behaviours and variables that help present the protocol in Section 3.4 and illustrate our findings in Section 4. In particular, our model retains

- essential network parameters,
- simplified data structures that retain fields relevant to quorum selection and service node state changes,
- the high-level execution flow of a service node, with concurrently running subsystems flattened into a linear execution order
- committee selection algorithms for Pulse and Obligation quorums
- the validation logic for state-change transactions,

We deliberately omit details on network communication, low-level data format, non-deterministic execution order induced by concurrency, and some implementation-level nuances that do not affect the attacks studied in this paper. For instance,

Block: We model a block as an abstract record containing only fields relevant to quorum mechanisms. We do not distinguish between block header and block body. `blockHash` denotes the canonical block hash derived from relevant fields of the record.

Transaction: We model each transaction as a record that retains only the fields used by other parts of the model. We keep the state-change record which contains the cached Obligation quorum height, worker, state-change operation, and validator votes as an abstract record. We omit other transaction fields such as transaction hash, fee, and block height.

Pulse: In the implementation, Pulse is executed by a long-lived worker that observes the current chain state and advances through a local state machine driven by timeouts and quorum messages. In the main body, we flatten this behaviour into $\mathsf{RunPulse}_\rho$, a single procedure that takes the current chain state as input, and yields the next accepted block.

The following appendix section describes helper functions of our model that are excluded from the main body.

C.2. Abstract Helper Functions

- $\mathtt{sns}' \leftarrow \mathsf{UpdateSNS}(\mathtt{sns} : \mathtt{SN[]}, \mathtt{txs} : \mathtt{Tx[]})$: applies the state-change transactions `txs` to the service node list `sns` and returns the resulting list $\mathtt{sns}'$. This includes updating node's active status according to the specified operation:

$$\text{do} \begin{cases} \mathtt{sn.active} \leftarrow \bot & \text{if DECOM} \\ \mathtt{sn.active} \leftarrow \top & \text{if RECOM} \\ \mathtt{sns} \leftarrow \mathtt{sns} \setminus \{\mathtt{sn}\} & \text{if DERGE,} \end{cases}$$

  and updating auxiliary fields used to compute remaining credits, which are omitted from our model.
- $\mathsf{UpdateLastPulse}(\mathtt{sns} : \mathtt{SN[]}, q : \mathtt{PQ}, i)$: Updates the `lastPulse` field of each `sn` participated in the quorum q to the current block height i.
- $\mathtt{votes} \leftarrow \mathsf{OblVoting}_{\rho,q}(\mathtt{vote})$: an abstraction of the Obligation quorum q's voting procedure that sends and receives the votes on the state-change proposal for worker $\mathtt{vote}.w$ and operation $\mathtt{vote.sc}$.
- $b \in \{\top, \bot\} \leftarrow \mathsf{ValidateVotes}(\mathtt{tx.votes}, S)$: an abstraction procedure that validates the votes for a state-change transaction.

C.3. Network parameters

- Cache delay:

```
reorgBuffer = (CHECKPOINT_INTERVAL *
    CHECKPOINT_NUM_CHECKPOINTS_FOR_CHAIN_FINALITY)
    + (CHECKPOINT_INTERVAL - 1)
    = (4 * 2) + (4 - 1)
    = 11
```

- Obligation quorum workers: The number of obligation quorum workers is computed as the maximum between `STATE_CHANGE_MIN_NODES_TO_TEST` and the available nodes divided by `STATE_CHANGE_NTH_OF_THE_NETWORK_TO_TEST`. Since Oxen currently has significantly fewer than 5000 nodes in the mainnet, the number of workers n_w in practice would always be 50.

```
STATE_CHANGE_NTH_OF_THE_NETWORK_TO_TEST = 100
STATE_CHANGE_MIN_NODES_TO_TEST = 50
STATE_CHANGE_MIN_VOTES_TO_CHANGE_STATE = 7
```

Appendix D. Simulation Set Up for the Network Takeover Attack

We built a lightweight Python simulator that mirrors Oxen's Pulse and Obligation quorum logic without introducing real-world non-determinisms (e.g. network latency, multi-threading), and implements our attack strategies for attacker-controlled quorums. The simulation code is organised into core modules:

- **data_types.py** Defines the core abstractions (e.g., ServiceNode, Quorum, Block, Transaction, State).
- **algorithms.py** Implements a distance-based combinatorial decommissioning search that allows an attacker-controlled Obligation quorum to find a set of honest worker nodes to decommission such that the next-next Pulse quorum will be predominated by attacker nodes. Given swapIndices, activeNodeList, honestWorkerIndices and a budget k (number of nodes to be decommissioned), it:
  - Computes, for each swap index i, the distance from activeNodeList[i] to the next dishonest node.
  - Greedily removes up to k honest test–nodes that lie within each local distance window, prioritising those closest to the swap indices.
  - If "local" removals alone don't push enough attackers into the next Pulse quorum, it runs a combinatorial search over earlier honest nodes to maximise the attacker count among the k removals.
  - Ensures exactly k nodes are removed (so that the subsequent swap indices remain consistent), and aborts early if the search times out.
- **oxen_attack.py** The main script:
  1) Initialise a network with active nodes and existing blocks with random data, marking a fraction p of nodes as malicious.
  2) Loop for T iterations, where each simulates one block time. During each iteration, the network:
     a) Quorum computation: The computation logic aligns with the core protocol's implementation. It computes the Pulse quorum from previous blocks' Pulse randomness; When a new block arrives, it computes the Obligation quorum from the block hash. The Obligation quorum is then stored in the State object.
     b) Run Pulse quorum: If the current Pulse quorum is attacker-controlled, it invokes a helper function that brute-forces different random values to find a Pulse randomness such that the block hash (from the block template that contains the Pulse randomness) would produce an attacker-controlled Obligation quorum. Otherwise, the

quorum operates honestly, adding a random Pulse randomness to the block being produced.

c) Run Obligation quorum: If the current Obligation quorum is attacker-controlled, it invokes the removal strategy as defined in algorithms.py over different k values, ranging from the number of honest worker nodes in the quorum to 0. The malicious quorum then submits decommission transactions based on the algorithm's selection to the network. Otherwise, the quorum is considered honest: it executes honestly and recommissions any decommissioned worker nodes.

d) Processing transactions: When a new block arrives, the network processes the transactions inside the block and updates the State object accordingly. For instance, decommissioning an active node if there is a decommission transaction for it.

Figure 7 visualises the workflow of the main simulation script (**oxen-attack.py**).

- **globals.py** Stores global parameters (e.g. quorum sizes, attacker fraction k, network size N, maximum iteration count, timeouts).

D.1. Simulation Limitations

Our simulation deliberately omits or simplifies several real-world aspects of the Oxen network:

- **Concurrency and network delay.** In the real Oxen client, Pulse and Obligation quorums are workers that run concurrently, and transaction propagation is subject to delays. Therefore, a state-change transaction produced by an Obligation quorum at height i may only reach other nodes' mempools at a later height, or the Pulse round state-machine may have already progressed beyond the Template stage (where the block producer populates the block template with transactions). In such cases, the transaction may not be included in block i, and may instead appear in block $i+1$ or later, which shifts the Pulse quorum composition from the prediction made by an OV1 attacker. Our simulation eliminates this timing variability by enforcing a strict sequential ordering, similar to $\mathsf{Oxen}_\rho(S_i)$ and assumes immediate transaction acceptance.
- **Pulse-quorum failures.** We assume a Pulse quorum always succeeds in producing a block. In reality, factors such as network partitioning, latencies, or clock drift, can cause a quorum to fail; omitting these failures keeps our focus on attack effectiveness.

Our simplifications make the simulation more optimistic (with respect to the adversary), since state-change transactions produced by Obligation quorums would be immediately added to the next block, and affect service node list used for subsequent quorum selection.

D.2. Simulation results

We modelled a 2000-node network with Pulse quorums of size 12; Obligation quorums of size 60 (10 validators and 50 workers), and the consensus threshold for

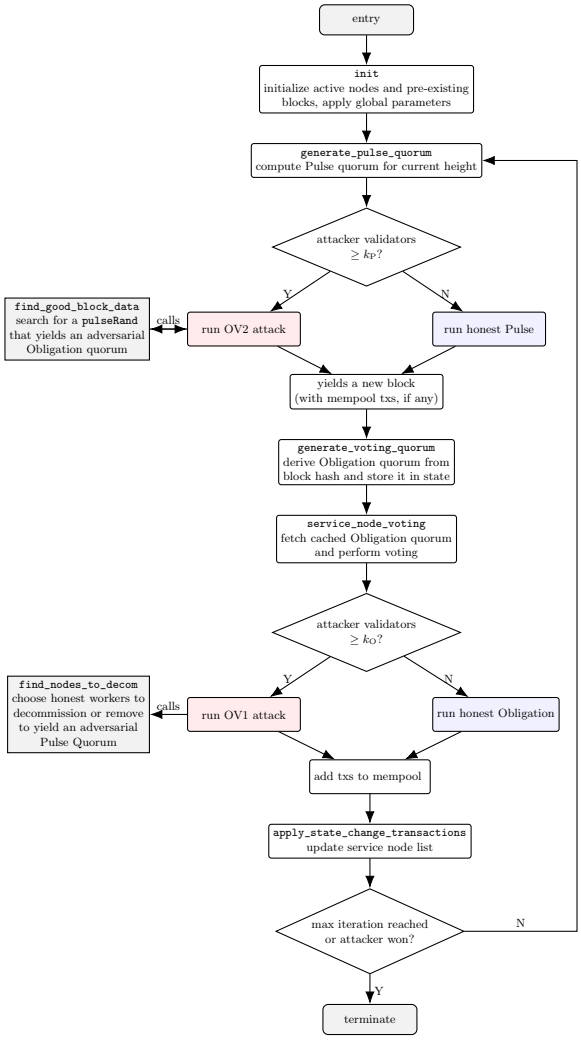


Figure 7: Oxen Simulation Workflow

both quorums is 7, matching the corresponding parameters used by the mainnet. We simulate an attacker controlling $p \cdot 2000$ nodes.

Figure 8 was captured from a single execution trace (with $p = 0.3$). Each line begins with the block height (H ℓ) at which it was emitted.

- **Purple lines:** produced by a malicious Obligation quorum at height ℓ. They report `RECOM`, `DECOM` and `DEREG` counts, the prediction for H $\ell+1$ Pulse-quorum swap indices (with attacker nodes in red, honest nodes in green), and the running "Success/-Total" ratio.
- **Green lines:** produced by the actual Pulse quorum elected at height $\ell+1$
- **White lines:** produced by the network when applying state change transactions from block ℓ. Only a malicious Obligation quorum emits decommission transactions, and only an honest one emits recommission transactions, so these entries reveal whether the Obligation quorums at the previous height was attacker-controlled.

We run the simulation with multiple seeds over different configuration parameters (see the artefact repo for

```
H 630 - failed to find a decommission comibination, decommission all honest workers instead Success/Total: 9/39
H 631 - recommissioned 0, decommissioned 29, removed 0
H 633 - recommissioned 8, decommissioned 0, removed 0
H 634 - pulse quorum (predict for H 635): [1792, 197, 283, 227, 1110, 134, 132, 510, 710, 919, 644, 169]  Success/Total: 10/40, decommed 16, removed 0
H 635 - recommissioned 0, decommissioned 16, removed 0
H 635 - pulse quorum (actual  for H 635): [1792, 197, 283, 227, 1110, 134, 132, 510, 710, 919, 644, 169]
H 643 - failed to find a decommission comibination, decommission all honest workers instead Success/Total: 10/41
H 644 - recommissioned 0, decommissioned 22, removed 0
H 650 - pulse quorum (predict for H 651): [1555, 177, 299, 554, 588, 433, 96, 1864, 81, 1594, 1892, 1302]  Success/Total: 11/42, decommed 17, removed 0
H 651 - recommissioned 0, decommissioned 17, removed 0
H 651 - pulse quorum (actual  for H 651): [1555, 177, 299, 554, 588, 433, 96, 1864, 81, 1594, 1892, 1302]
H 657 - recommissioned 14, decommissioned 0, removed 0
H 663 - pulse quorum (predict for H 664) is already controlled by attackers
H 667 - recommissioned 8, decommissioned 0, removed 0
H 669 - pulse quorum (predict for H 670): [1327, 1836, 182, 537, 228, 831, 729, 686, 431, 451, 12, 329]  Success/Total: 12/43, decommed 17, removed 0
H 670 - recommissioned 0, decommissioned 17, removed 0
H 670 - pulse quorum (actual  for H 670): [1327, 1836, 182, 537, 228, 831, 729, 686, 431, 451, 12, 329]
H 672 - recommissioned 14, decommissioned 0, removed 0
H 676 - pulse quorum (predict for H 677): [1499, 453, 207, 1233, 835, 154, 1179, 445, 2, 466, 251, 1769]  Success/Total: 13/44, decommed 2, removed 0
H 677 - recommissioned 0, decommissioned 2, removed 0
H 677 - pulse quorum (actual  for H 677): [1499, 453, 207, 1233, 835, 154, 1179, 445, 2, 466, 251, 1769]
H 684 - pulse quorum (predict for H 685): [214, 675, 1018, 464, 219, 560, 173, 353, 1163, 561, 1376, 158]  Success/Total: 14/45, decommed 3, removed 0
H 685 - recommissioned 0, decommissioned 3, removed 0
H 685 - pulse quorum (actual  for H 685): [214, 675, 1018, 464, 219, 560, 173, 353, 1163, 561, 1376, 158]
H 690 - pulse quorum (predict for H 691) is already controlled by attackers
H 691 - pulse quorum (predict for H 692) is already controlled by attackers
H 695 - pulse quorum (predict for H 696): [1539, 113, 155, 90, 1040, 92, 398, 20, 1904, 703, 377, 1354]  Success/Total: 15/46, decommed 4, removed 0
H 696 - recommissioned 0, decommissioned 4, removed 0
H 696 - pulse quorum (actual  for H 696): [1539, 113, 155, 90, 1040, 92, 398, 20, 1904, 703, 377, 1354]
```

Figure 8: Observed Consecutive success of Quorum-Chaining Attack

more details and the raw output):

```
PULSE_QUORUM_SIZE = 12
PULSE_MIN_SIG = 7
VOTING_QUORUM_SIZE = 10
VOTING_WORKER_SIZE = 50
VOTING_MIN_SIG =7
DECOM_THRESHOLD = 1000
TOTAL_NODES = 2000
NUM_EXISTING_BLOCKS = 100
MAX_ITERATION = 10000
PULSE_TIME_OUT = 5
OBL_TIMEOUT=5
VOTE_LIFE = 65
...
ADVERSARY_CONFIG = <p>
RANDOM_SEED = <seed>
REORG_BUFFER = <reorgBuffer>
<Flags to enable/disable each OV>
```

OV1,2 - Baseline. Table 4 shows the blocks taken to take over the network by exploiting OV1 and OV2 as a baseline

TABLE 4: Blocks required to win OV1,2

p	seed "1"	"12"	"123"	"1234"	"12345"	median
0.25	Failed	Failed	Failed	Failed	Failed	Failed
0.26	2700	3400	3700	3300	2100	3300
0.27	1600	2000	2300	1900	1700	1900
0.29	1500	1300	1600	1400	1300	1400
0.31	1300	1000	1100	1200	1400	1200
0.33	1000	1100	900	900	1100	1000

OV1,2,3.1 - Early voting. Table 5 shows the result of exploiting OV1,2,3.1 to show the effect of OV3.1, where an adversarial Obligation quorum disregards the intended delay induced by `reorgBuffer`, and votes early.

TABLE 5: Blocks required to win OV1,2,3.1

p	seed "1"	"12"	"123"	"1234"	"12345"	median
0.26	2600	4100	1700	2600	2200	2600
0.27	2100	1700	2100	2100	2000	2100
0.29	1500	1400	1300	1300	1300	1300
0.31	1200	1100	1000	1100	1200	1100
0.33	900	900	900	1100	900	900

OV1,2,3.2 - Revoting. Table 6 shows the result of exploiting OV1,2,3.2 to show the effect of OV3.2, where historical adversarial Obligation quorums revote to amplify their influence on future Pulse selection.

TABLE 6: Blocks required to win OV1,2,3.2

p	seed "1"	"12"	"123"	"1234"	"12345"	median
0.26	800	1600	1300	1100	800	1100
0.27	800	700	1100	800	600	800
0.29	500	700	600	700	700	700
0.31	500	600	500	500	500	500
0.33	500	400	400	400	400	400

OV1,2,4 - Immediate Deregistration. Table 7 shows the result of exploiting OV1, 2, 4 to show the effect of OV4.

TABLE 7: Blocks required to win OV1,2,4

p	seed "1"	"12"	"123"	"1234"	"12345"	median
0.21	5000	3100	4100	3700	3400	3700
0.23	2700	2000	3100	2800	1700	2700
0.25	1600	1600	1700	1800	1300	1600
0.27	900	1100	1200	1000	900	1000
0.29	900	900	900	800	800	900

Mitigation Effectiveness. Table 8 shows the effectiveness of Oxen's mitigation (discussed in Sec. 7.2), where OV4 is patched and the Pulse quorum size is increased ($n_{\text{Pulse}} = 25$, $k_{\text{P}} = 16$), while the rest of the vulnerabilities persist. We analyse the effectiveness of this mitigation against an adversary who exploits OV1,2,3.2.

TABLE 8: Blocks required to win OV1,2,3.2 with a larger Pulse quorum ($n_{\text{Pulse}} = 25$, $k_{\text{P}} = 16$)

p	seed "1"	"12"	"123"	"1234"	"12345"	median
0.30	Failed	Failed	Failed	8800	Failed	Failed
0.31	2100	1600	4400	2900	1600	2100
0.32	1500	1000	1600	1200	1100	1200
0.33	900	1100	600	1100	1800	1100